AGENTIC SECURITY AUDIT REPORT

HookRig

Composable Uniswap v4 hook orchestrator & token launchpad

Report date	2026-10-15
Audit iterations	4 scans · 9 passes · 108 specialist-agent runs
Items raised	236 (206 findings, 30 leads)
Current status	158 closed (fixed, mitigated, obsolete or by design) · 78 open
Code reviewed	36 Solidity files (src/ and script/), commits 8477cac → 594edcd + working tree
Method	pashov solidity-auditor v4.1 (12 parallel specialist agents per pass, Claude Opus)

Confidential working document. Prepared from the run files in `.solidity-auditor/runs/` and a verification of every item against the code on disk on 2026-10-15.

1. Executive Summary

Four agentic audit scans of the HookRig repository raised 236 items (206 findings above or below the 75-point confidence threshold and 30 leads). Each scan was followed by a remediation round. Every item from every scan was then checked again against the current code: 135 are fixed, 2 are mitigated, 10 are obsolete because the code path no longer exists or is unreachable, and 11 are accepted design behaviour. 78 items remain open (34 partially fixed, 44 not fixed); they are grouped into 16 work items in Section 4.

Many open items are the same bug raised again by a later scan, so the count of distinct open issues is much smaller than the item count. The most important open item is TODO-22: the AutoBuyback floor-tick comparison is still inverted for both token orders, and three scans raised it.

1.1 Status by iteration

Scan	Items	FIXED	MITIGATED	OBSOLETE	BY-DESIGN	PARTIAL	PRESENT	PENDING	Open
S1	36	28	–	1	–	5	2	–	**7**
S2	68	36	–	5	4	10	13	–	**23**
S3	75	39	–	4	4	17	11	–	**28**
S4	57	32	2	–	3	2	–	18	**20**
Total	**236**	**135**	**2**	**10**	**11**	**34**	**26**	**18**	**78**

1.2 Key outcomes

- Critical claim-path issues are closed: limit-order claim swaps now run the full hook pipeline (fees, cuts, protections), are attributed to the maker, are reentrancy-guarded and accept a minimum output.
- The launchpad no longer accepts creator-written hooks, AccessControl on launch pools, AntiSnipe launch fees above 25% or invalid tick spacing, and CREATE2 salts are bound to the sender.
- LP donations (ArbRecapture, LPRewards, AutoBuyback) are always taken and paid out on a later swap in a block without liquidity adds, which closes same-transaction and same-block capture.
- TWAMM order submission is disabled because the block never swaps deposits; real execution is tracked as TODO-14.
- Open work is concentrated in launch-curve economics (TODO-26), untrusted-router identity (TODO-24), the JIT guard design (TODO-16), the inverted AutoBuyback floor (TODO-22) and owner-trust hardening (TODO-15).

1.3 Status definitions

FIXED	The defect no longer exists in the current code.
MITIGATED	The vulnerable path is disabled; the underlying feature still needs work.
OBSOLETE	The function or path was removed or is unreachable.
BY-DESIGN	Intentional, documented behaviour (docs/04 §3.7, docs/07 TODO-21).
PARTIAL	Mitigated, but part of the described issue remains.
PRESENT	An earlier finding that still exists in the current code.
PENDING	A finding or lead from the latest scan that is not yet fixed.

2. Scope & Methodology

2.1 Scope

All Solidity production code was in scope: `src/` (orchestrator, base aggregators, 22 blocks, libraries, presets, launchpad factory, token and three launch curves) and the deploy script `script/HoodiIntegration.s.sol`. Dependencies (`lib/`), interfaces, mocks and tests were out of scope.

2.2 Agentic audit process

Each pass runs 12 specialist agents in parallel over the full source bundle. Agents 1–9 each attack from one angle; agents 10–12 hunt the seams between angles:

#	Specialist	#	Specialist	#	Specialist
1	Math precision	2	Access control	3	Economic security
4	Execution trace	5	Invariants	6	Periphery
7	First principles	8	Asymmetry	9	Boundary
10	Numerical gap	11	Trust gap	12	Flow gap

- Deduplication: findings are grouped by contract, function and bug class; fixes with different code are kept as separate options.
- Four-gate judging: attack execution, reachability, unprivileged trigger and material impact. Admin-only harm needs an unprivileged amplifier. Confidence starts at 100 and is reduced for partial paths, bounded impact and special state. The threshold is 75.
- Memory ledger: from scan 2 onward, a ledger (`.solidity-auditor/memory.tsv`) passes earlier findings to each pass so later passes look at new ground. Scan 3 ran as a single pass without memory.
- Multi-pass loops: every pass after the first is told what earlier passes found; `seen in k/N runs` in the run reports shows how many passes raised an item.
- Final verification (this report): six independent read-only agents re-checked every scan 1–3 item against the code on disk; scan 4 items were fixed and tested in this session.

2.3 Audit iterations

Scan	Started	Commit	Passes	Findings	Leads	Remediation	Ledger
S1	2026-10-14 21:57	`8477cac`	2	36	0	651b515 (feat: first agent audit run)	0 → 36
S2	2026-10-15 03:48	`651b515`	3	66	2	b32641f, ede9c74 (second agent audit pass)	26 → 68
S3	2026-10-15 07:18	`ede9c74`	1	69	6	b0859d2 (feat: audit pass 3 and fuzz testing)	not used
S4	2026-10-15 10:03	`594edcd`	3	35	22	working tree (uncommitted, this session)	66 → 110

Run files: `.solidity-auditor/runs/20261014-215713`, `20261015-034829`, `20261015-071849` and `20261015-100328`. Item IDs in this report are `<scan>-F<n>` for findings and `<scan>-L<n>` for leads, numbered as in each scan's full report.

3. Latest Remediation (Scan 4)

Scan 4 raised 35 findings and 22 leads. 32 findings were fixed in the working tree, with 16 new regression tests in `test/unit/AuditFixes.t.sol`. The full suite (87 local tests plus 3 Hoodi fork tests) passes. The orchestrator keeps 1,853 bytes of runtime size margin. Design decisions taken with the project owner:

- TWAMM (#32, #33): new orders are disabled instead of shipping a half-built execution path; existing orders stay cancellable for a full refund.
- Curve LP fees (#22, #30): the curve-owned graduation LP stays locked, like burned LP tokens; no new withdrawal path that could reward creators for high fees.

- LP donations (#11, #12, #14): cuts are always taken, queued as hook ERC-6909, and donated in a later swap's beforeSwap in a block with no liquidity add.
- Sell caps (#24, #34): one-sided liquidity is rejected on WalletCap and VestingSellCap pools; wallet splitting is documented as an inherent limit of per-wallet caps.
- JIT guard (#8): not changed. Keying the hold by position would let any user of a shared router re-lock everyone's liquidity, and the bypass also works with two separate positions (TODO-16).

Behaviour changes for integrators

- `claimLimitOrder(key, orderId, minAmountOut)` added; a filled but unclaimed order can be cancelled.
- `predictTokenAddress` is sender-bound; use `predictTokenAddressFor(meta, salt, deployer)`.
- Exact-output buys revert on pools with AutoBurn or King of the Pool; taxes can only decrease after registration.
- Pools can only be registered for tokens that already exist; preset hooks need new address flags (redeploy).

4. Pending Work Register

Every open item (78 PARTIAL, PRESENT or PENDING) maps to exactly one work item below. The same IDs are used in `docs/07-roadmap-and-todo.md`. TODO-09 (live Hoodi broadcast) is a deployment task and is not repeated here.

ID	Priority	Work item	Related items	Status
TODO-14	P1 (High)	**Real TWAMM Execution** `executePending` only accrues virtual volume and never swaps deposits, so `submitTWAMMOrder` is disabled (`TWAMMOrdersDisabled`). Implement in-`beforeSwap` execution of accrued sell volume, per-order output accounting and a claim path, then re-enable submission with a minimum order size so dust orders cannot fill `MAX_ACTIVE_TWAMM_ORDERS`. Target: TWAMMBlock.sol	S4-F32, S4-F33	Disabled (Orders Rejected)
TODO-15	P1 (High)	**Owner-Trust Hardening of setLaunchpadFactory** The owner can point `launchpadFactory` at any address, which may then seed King pots without ERC-6909 backing (burning other pools' shared claims) and restart graduation clocks on standard pools. Make the factory settable once (or timelocked) and verify the minted backing in `seedKingPotFromLaunchCurve`. Target: HookRigBase.sol, KingOfThePoolBlock.sol	S4-L05, S4-L07, S4-L09	Open
TODO-22	P1 (High)	**Inverted AutoBuyback Floor-Tick Comparison** `accumulateAndCheckTrigger` uses >= when the project token is token0 and <= when it is token1, the reverse of a price drop in both orders, so floor defense fires when the price is above the floor. Flip both comparisons and add a test for each token order. Target: AutoBuybackBlock.sol	S1-F05, S2-F04, S3-F14	Open (raised in 3 scans)
TODO-16	P2 (Low)	**JIT Guard Key Design & Exit Penalty** Hold periods are keyed per end user only for trusted-router salt-0 positions; untrusted routers and non-zero salts share one key that any 1-wei add can reset (re-locking others up to 7,200 blocks), and an attacker who adds from one address and removes an older position from another escapes the hold. `jitExitPenaltyBps` is read but never deducted, although docs/04 says it is enforced. Needs one design that blocks both the bypass and the re-lock, plus a real penalty or corrected docs. Target: JITGuardBlock.sol	S4-F08, S3-F15, S3-F16, S3-F17, S3-F18, S3-F19, S3-F20, S2-L01, S3-L03	Known Limit
TODO-17	P2 (Low)	**Fee-on-Transfer Quote Tokens** `buy` records the requested quote, not the received amount, so a fee-on-transfer quote token makes graduation revert and the last sellers find the balance short. Add balance-delta accounting or reject such quote tokens in the factory. Target: VirtualXYKLaunch.sol, SteppedTrancheLaunch.sol	S3-F69, S4-L18, S4-L21	Open
TODO-18	P2 (Low)	**Deploy Script Key Fallback** `vm.envOr("PRIVATE_KEY", <Anvil key #0>)` falls back to a public key; anyone who knows it would own a broadcast deployment. Require `PRIVATE_KEY` (no default) before TODO-09. Target: script/HoodiIntegration.s.sol	S4-L03	Open
TODO-19	P2 (Low)	**Integrator Notes & Preset Redeploy** Document the `claimLimitOrder(key, orderId, minAmountOut)` overload, `predictTokenAddressFor(meta, salt, deployer)` (salts are sender-bound), exact-output buys reverting on AutoBurn / King pools, queued LP donations and decay-only taxes in docs 02/03/05/06; re-mine and redeploy preset hooks whose permission flags changed (e.g. FairLaunch now needs `beforeAddLiquidity`). Target: docs/, README.md	–	Open

ID	Priority	Work item	Related items	Status
TODO-23	P2 (Low)	**Standard-Pool Initialization & First-Come Registration** `_beforeInitialize` checks the caller only for launchpad pools, so anyone can initialize a registered standard pool at any price; standard pools of existing tokens are still first-come with `admin = msg.sender`. Restrict initialization to the pool admin (or register + initialize atomically) and document first-come registration. Target: HookRigOrchestrator.sol, AccessControlBlock.sol	S2-F63, S3-F58, S2-F24, S3-F41	Open
TODO-24	P2 (Low)	**Untrusted-Router Identity** For untrusted routers the resolved trader is the router: WalletCap checks the router's balance (buyers pass the cap every transaction), King can crown the router (the pot becomes unclaimable), `registerReferrer` binds `msg.sender` with no router check, an allowlisted public router admits every caller, and a router that pre-settles still trips the vesting cap. Decide a per-block policy (require trusted routers, skip, or revert) and apply it consistently. Target: AccessControlBlock.sol, WalletCapBlock.sol, KingOfThePoolBlock.sol, ReferralSplitBlock.sol, VestingSellCapBlock.sol	S3-F34, S3-F35, S3-F36, S4-L22, S2-F64, S2-F65, S2-F47, S4-L01, S3-F33	Open
TODO-25	P2 (Low)	**Dynamic Fee Lags the Swap That Moves the Price** `_resolveBeforeSwapFee` derives volatility from the pre-swap tick, so a whale's own price move raises the fee for the traders after it rather than for the whale. Consider charging on the realized move (e.g. an afterSwap surcharge) or documenting the lag. Target: DynamicFeeBlock.sol	S1-F08	Open
TODO-26	P2 (Low)	**Launch Curve Economics** Creator vesting starts at deploy + 1h, not at graduation, so vested tokens can be sold into an active curve; the stepped curve caps `graduationQuoteTarget` at one tranche, prices a whole order at one tranche price and redeems sells at the base price; buys can return zero tokens; `invariantK` is never refreshed; `antiSnipeBlocks` is unbounded and surcharges have no refund in a stalled launch; a curve wallet cap above 200 bps with pool cap 0 reverts; the stepped curve clamps LP tokens without scaling quote when surcharges are large. Target: LaunchpadFactory.sol, SteppedTrancheLaunch.sol, VirtualXYKLaunch.sol	S1-F18, S2-F25, S3-F21, S2-F10, S3-F22, S1-F25, S2-F13, S3-F29, S1-F27, S2-F19, S2-F52, S2-F58, S3-F64, S2-F09, S4-L17	Open
TODO-20	P3 (Info)	**Remaining Audit Leads** Unverified, no proven loss: AutoBurn burns from the hook's shared balance if a token under-delivers on `take`; the Hoodi script router can spend ETH it already holds; exact-output sells earn fewer swap points; a 1-wei order can clear a filled bucket and delay claims; zeroForOne orders fill on a downward crossing (confirm stop-order semantics); swaps over 512 tick steps leave later buckets unfilled; the crossed-tick loop starts at the previous tick; a hook without afterSwap would keep a stale snapshot; ArbRecapture is not counted in `validateTaxBudget`. Target: various	S4-L02, S4-L04, S3-L02, S4-L06, S4-L13, S4-L15, S4-L16, S1-F21, S3-L05, S3-L04, S4-L19	Open
TODO-21	P3 (Info)	**Accepted Design Limits** Per-wallet sell caps cannot stop a holder who splits tokens across wallets (needs a token-level transfer hook), and curve-owned graduation LP fees and leftover donations stay locked in PoolManager by design (docs/04 §3.7). Revisit if either becomes a product requirement. Target: VestingSellCapBlock.sol, launch curves	S2-F17, S4-F34, S4-F22, S4-F30	Accepted
TODO-27	P3 (Info)	**NativeTick Leftover Supply** The factory sends the full non-vested supply to `NativeTickLaunch`, but bands may use less, and zero, invalid or zero-liquidity bands are skipped; the leftover tokens have no sweep. Validate band totals and liquidity in the factory, or add a sweep to the creator / burn. Target: NativeTickLaunch.sol, LaunchpadFactory.sol	S2-F41, S2-F46, S3-F63, S2-F62	Open

ID	Priority	Work item	Related items	Status
TODO-28	P3 (Info)	**Protection-Block Tuning** A swap of at least `dirThreshold` ticks still sets the block direction and blocks opposite trades of that size; the CircuitBreaker anchor can drift one band per window; ArbRecapture lets earlier sub-threshold moves add to a later trader's cut (intended anti-split); external flash loans can still inflate the first-of-epoch vesting snapshot; King `minBuyAmountForReset` may be unreachable for small-supply tokens, and with a 0 minimum small non-dust buys crown just before the deadline. Target: MEVProtectionBlock.sol, CircuitBreakerBlock.sol, ArbRecaptureBlock.sol, VestingSellCapBlock.sol, KingOfThePoolBlock.sol	S2-F27, S3-F61, S3-F62, S1-F07, S3-F68, S2-F03, S3-F30, S2-F39, S3-F44, S4-L08	Open
TODO-29	P3 (Info)	**Admin & Configuration Ergonomics** `setManagedPoolFee` reverts before the pool is initialized; `setBuySellTax` reverts on a zero recipient even to keep the current one while any tax is non-zero; a zero-creator launch makes the factory the pool admin; VestingSellCap accepts caps up to 10,000 bps against the documented 5,000. Target: ManagedFeeBlock.sol, BuySellTaxBlock.sol, LaunchpadFactory.sol, VestingSellCapBlock.sol	S2-F45, S2-F55, S4-L10, S4-L20	Open

5. Scan 1 — 2026-10-14 21:57 (8477cac)

2-pass scan of commit `8477cac` (SA-01 – SA-36). Remediation: 651b515 (feat: first agent audit run). 36 findings and 0 leads; current status: 28 fixed, 1 obsolete, 5 partial, 2 present.

ID	Conf.	Finding	Status	Current evidence
S1-F01	95	**AccessControlLogic.verifyAccess requires hookData.length >= 64 and rejects single-leaf Merkle trees** `AccessControlBlock._verifyAccess` · seen in 1/2 runs · NEW `AccessControlLogic.verifyAccess` requires at least 64 bytes of `hookData`, so a single-leaf Merkle allowlist with an empty proof always reverts.	FIXED	verifyAccess now returns early on `leaf == root` (single-leaf tree) and accepts proofs of >=32 bytes (AccessControlBlock.sol:193-207).
S1-F02	95	**A buyer who submits an exact-output swap skips AutoBurnBlock and KingOfThePoolBlock fees** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` · seen in 2/2 runs · NEW A buyer who submits an exact-output swap skips `AutoBurnBlock` and `KingOfThePoolBlock` fees because `unspecifiedCurrency` is the quote token.	FIXED	_executeAfterSwapDeltaWaterfall reverts ExactOutputBuyNotSupported for exact-output buys when AutoBurn or King is enabled (AfterSwapDeltaAggregator.sol:60-65).
S1-F03	95	**_validateAggregateReturnDeltaBudget omits BIT_ARB_RECAPTURE and counts zero-liquidity LP donations against the cap** `AfterSwapDeltaAggregator._validateAggregateReturnDeltaBudget` · seen in 2/2 runs · NEW When post-swap liquidity is zero, `ArbRecapture` and `LPRewards` fill `maxAllowedTotalCut` before `totalCutAmount` drops `lpDonationAmount`, so later fee blocks get zero cut.	FIXED	LP donations are always queued (queueLPDonation), never dropped from totalCutAmount at zero liquidity (docs/04:87). Excluding ArbRecapture from the budget is documented in docs/06:57.
S1-F04	95	**computeCutBps reads state.referenceTick before it updates referenceTick for a new block** `ArbRecaptureBlock.computeCutBps` · seen in 2/2 runs · NEW `computeCutBps` updates `state.referenceTick` after it reads `refTick`, so the first swap in the next block pays up to 10% extra fee.	FIXED	computeCutBps updates referenceTick=preSwapTick on a new block before reading refTick (ArbRecaptureBlock.sol:47-52).
S1-F05	95	**accumulateAndCheckTrigger checks postSwapTick <= triggerFloorTick even when projectToken is currency1** `AutoBuybackBlock._processAutoBuyback` · seen in 2/2 runs · NEW The `accumulateAndCheckTrigger` function checks `postSwapTick <= triggerFloorTick` even when the project token is `currency1`, where price drops move the tick up instead of down.	PRESENT	AutoBuybackBlock.sol:82-83 still uses >= when the project token is token0 and <= when it is token1. Both are the reverse of a price drop, which moves the tick down for token0 and up for token1. → TODO-22
S1-F06	95	**HookRigOrchestrator exposes no external function for _updateBuySellTax and update rejects zero newRecipient** `BuySellTaxBlock._updateBuySellTax` · seen in 2/2 runs · NEW The pool admin cannot lower taxes in `BuySellTaxBlock`, because `HookRigOrchestrator` exposes no external function for `_updateBuySellTax` and `BuySellTaxLogic.update` rejects a zero `newRecipient`.	FIXED	HookRigOrchestrator.sol:169 setBuySellTax exposes _updateBuySellTax to poolAdmin; update only rejects zero recipient when the new taxes are nonzero (BuySellTaxBlock.sol:107).
S1-F07	95	**CircuitBreakerLogic.checkAndAdvance anchors new windows to preSwapTick and lets an attacker lock out one swap direction** `CircuitBreakerBlock._checkAndAdvanceCircuitBreaker` · seen in 1/2 runs · NEW An attacker swaps to the band edge before window expiry and unwinds in the new window to lock `anchorTick` and block sells.	PARTIAL	On rollover, anchor = preSwapTick clamped to +/-maxWindowTickDeviation of the previous anchor (CircuitBreakerBlock.sol:54-64). An attacker can still drift the anchor one band per window. → TODO-28

ID	Conf.	Finding	Status	Current evidence
S1-F08	95	**DynamicFeeLogic.compute reads preSwapTick before the swap moves the price and spikes intra-block volatility without EMA damping** `DynamicFeeBlock._computeDynamicFee` · seen in 1/2 runs · NEW A whale who executes the first swap in a block pays `baseFee` on a large tick move and shifts `maxFee` onto subsequent traders.	PRESENT	_beforeSwap still calls _resolveBeforeSwapFee(poolId,bitmap,preTick). DynamicFeeLogic.compute uses the pre-swap tick, so a whale's own move is charged to later traders. → TODO-25
S1-F09	95	**permissionsForBitmap omits callback flags for beforeInitialize, afterInitialize, beforeSwap, and afterSwap** `HookPermissionMasks.permissionsForBitmap` · seen in 2/2 runs · NEW `permissionsForBitmap` omits callback flags, so an attacker can initialize launchpad pools early on preset hooks and make graduation fail forever.	FIXED	permissionsForBitmap now sets afterInitialize, beforeSwap and afterSwap. validateBitmapAndPermissions checks the hook address flags. The afterInitialize guard blocks unauthorized init.
S1-F10	95	**An attacker passes a chosen address in hookData from an untrusted router to bypass access and cap rules** `HookRigBase._resolveUser` · seen in 2/2 runs · NEW An attacker passes a chosen address in `hookData` from an untrusted router, so `_resolveUser` returns that address and bypasses access and cap rules.	FIXED	HookRigBase._resolveUser delegates to AccessControlLogic.resolveUser, which returns routerSender for untrusted routers and never reads hookData.
S1-F11	95	**_beforeAddLiquidity enforces AccessControlBlock on authorizedCurve and blocks graduation liquidity** `HookRigOrchestrator._beforeAddLiquidity` · seen in 2/2 runs · NEW `_beforeAddLiquidity` checks `AccessControlBlock` even when `sender` is `authorizedCurve`, so a launch curve with access control fails when it adds liquidity.	FIXED	_beforeAddLiquidity skips _verifyAccess when `isCurve` (sender == authorizedCurve on a launchpad pool) (HookRigOrchestrator.sol:257-261).
S1-F12	95	**notifyPoolGraduated resets AntiSnipeBlock startBlock but leaves WalletCapBlock and KingOfThePoolBlock clocks unreset** `HookRigOrchestrator.notifyPoolGraduated` · seen in 2/2 runs · NEW A buyer after launchpad graduation can bypass `WalletCapBlock` and trade in an expired `KingOfThePoolBlock` round, because `notifyPoolGraduated` resets only `AntiSnipeBlock`.	FIXED	AntiSnipeLogic.resetGraduationClocks (AntiSnipeBlock.sol:90-98) now also resets the WalletCap startBlock and the King roundDeadline.
S1-F13	95	**An attacker can front-run registerPoolStack or PoolManager.initialize to take control or block graduation** `HookRigOrchestrator.registerPoolStack` · seen in 2/2 runs · NEW An attacker calls `registerPoolStack` or `initialize` before a launch, so the attacker takes control of the pool settings or makes graduation fail.	FIXED	initPoolMeta: a launchpad config requires msg.sender==launchpadFactory (AccessControlBlock.sol:85) and both tokens deployed (:99); _beforeInitialize reverts on unconfigured pools.
S1-F14	95	**HookRigOrchestrator._runAfterSwapPipeline uses params.amountSpecified instead of settled swapDelta on partial fills** `HookRigOrchestrator._runAfterSwapPipeline` · seen in 1/2 runs · NEW A partial swap stopped by `sqrtPriceLimitX96` passes the full `params.amountSpecified` to `VestingSellCapBlock` and `WalletCapBlock`, which overcounts the trade.	FIXED	_runAfterSwapPipeline uses ctx.settledSpecifiedAmount/grossUnspecifiedAmount from swapDelta (PoolTokenDir.sol:46-56), not params.amountSpecified (Orchestrator:387-405).

ID	Conf.	Finding	Status	Current evidence
S1-F15	95	**HookRigOrchestrator.seedKingPotFromLaunchCurve diverts buyer anti-snipe surcharges to poolAdmin when KingOfThePoolBlock is disabled** `HookRigOrchestrator.seedKingPotFromLaunchCurve` · seen in 1/2 runs · NEW A token creator who disables `KingOfThePoolBlock` on a stepped tranche launch receives the buyer anti-snipe surcharges in `poolClaims`.	FIXED	seedPotFromLaunchCurve reverts KingPotSeedNotAccepted unless KING is enabled and potFeeOnBuyBps>0 (KingOfThePoolBlock.sol:264-268); it never credits poolAdmin.
S1-F16	95	**JITGuardLogic.recordAddition keys positions by _resolveUser and overwrites lpAddedBlock on any 1-wei addition** `JITGuardBlock._recordLPAddition` · seen in 1/2 runs · NEW An attacker passes a different or victim address in `hookData` during `modifyLiquidity` to bypass the hold period or reset a victim cooldown timer.	FIXED	resolveUser ignores hookData and trusts only the router's msgSender (AccessControlBlock.sol:238-251); recordAddition overwrites unconditionally (JITGuardBlock.sol:27), so victim addresses cannot be spoofed.
S1-F17	95	**processBuy overwrites potCurrency from currency0 to currency1 and sums both currencies into currentPotAmount** `KingOfThePoolBlock.processBuy` · seen in 2/2 runs · NEW A post-graduation buy overwrites `potCurrency` from `currency0` to `currency1` while `currentPotAmount` holds `currency0`, so `claimKingPot` fails for the round winner.	FIXED	KingOfThePoolBlock.sol:142-145 only sets potCurrency when currentPotAmount==0 and adds only when potCurrency==cutCurrency. Aggregator:156-158 restricts King to the project token.
S1-F18	95	**_deployTokenAndVest allows pre-graduation vested token dumps and unvalidated curve supply allocations** `LaunchpadFactory._deployTokenAndVest` · seen in 2/2 runs · NEW A creator releases vested tokens during the bonding phase and sells them into the curve to take buyer quote tokens, or sets allocations above post-vesting supply.	PARTIAL	Allocations validated (LaunchpadFactory.sol:93-97,127-131). Vesting still starts at block.timestamp+1h (line 327), with no graduation gate, so creator can sell vested tokens into an active curve. → TODO-26
S1-F19	95	**Launch curves call notifyPoolGraduated on immutable hookOrchestrator instead of poolKey.hooks** `LaunchpadFactory.launchVirtualXYK` · seen in 2/2 runs · NEW The launch curves call `notifyPoolGraduated` on the immutable `hookOrchestrator` instead of `poolKey.hooks`, so any launch that uses a preset hook fails at graduation.	FIXED	Curves call notifyPoolGraduated on poolKey.hooks (VirtualXYKLaunch.sol:220, SteppedTrancheLaunch.sol:214/255, NativeTickLaunch.sol:58). _resolveHook only accepts defaultOrchestrator (Factory:192-196).
S1-F20	95	**placeOrder collects no ETH when inputCurrency is address zero and lets callers drain contract ETH** `LimitOrdersBlock.placeLimitOrder` · seen in 2/2 runs · NEW `placeOrder` accepts a native ETH order without ETH payment, so any caller can place and cancel an order to take all ETH.	FIXED	LimitOrdersBlock.sol:193-194 placeOrder requires `msg.value == inputAmount` for native input (and msg.value==0 for ERC20, L196).
S1-F21	95	**processCrossedTicks truncates toward zero, skips ticks past 16 steps, and never resets bucket.filled** `LimitOrdersBlock.processCrossedTicks` · seen in 2/2 runs · NEW A swap in `LimitOrdersBlock` fills uncrossed ticks near zero and skips ticks past 16 steps, because `processCrossedTicks` truncates toward zero and overwrites `lastTicks`.	PARTIAL	Floor/ceil alignment fixed (L348-349,360-361); filled reset in place/cancel/claim. Cap is now 512 steps (L120,351); ticks past cap still skipped. → TODO-20
S1-F22	95	**MEVProtectionLogic.enforceAfterSwap writes lastSwapBlock and lastSwapZeroForOne without reading them** `MEVProtectionBlock.enforceAfterSwap` · seen in 2/2 runs · NEW An attacker can split a large swap or execute a same-block sandwich in `MEVProtectionBlock`, because `enforceAfterSwap` writes `lastSwapBlock` and `lastSwapZeroForOne` but never reads them.	FIXED	enforceAfterSwap now reads lastSwapBlock/blockStartTick for cumulative block delta and lastSwapZeroForOne for direction (MEVProtectionBlock.sol:60-85).

ID	Conf.	Finding	Status	Current evidence
S1-F23	95	**NativeTickLaunch.unlockCallback passes bands[i].tokenAmount directly as liquidityDelta** `NativeTickLaunch.unlockCallback` · seen in 2/2 runs · NEW The callback passes the raw `tokenAmount` directly as `liquidityDelta`, so `PoolManager` deposits the wrong token amount and leaves supply inside the curve contract.	**FIXED**	NativeTickLaunch.unlockCallback:83-85 converts tokenAmount with LiquidityAmounts.getLiquidityForAmount1 and skips liq==0.
S1-F24	95	**updateCheckpoint truncates elapsed * MAX_ORACLE_TICK_DELTA_PER_SEC to int24 and wraps negative** `OracleBlock.updateCheckpoint` · seen in 2/2 runs · NEW After 83,887 seconds without a checkpoint, `int24(maxChange)` overflows and makes every swap, deposit, and withdrawal fail forever.	**FIXED**	OracleBlock.sol:112-127 _clampTick computes maxChange in int256 and clamps to [MIN_TICK,MAX_TICK] before the int24 cast.
S1-F25	95	**SteppedTrancheLaunch.buy prices the full order at the current tranche price and skips higher tranches** `SteppedTrancheLaunch.buy` · seen in 2/2 runs · NEW `buy` prices the full order at the current tranche price, so one buyer can buy the whole supply at the lowest price.	**PARTIAL**	buy still prices whole order at one currentMarginalPriceQ96 (Stepped:128-131), but factory caps target at one tranche (Factory:132-143), which bounds the purchase. → TODO-26
S1-F26	95	**SteppedTrancheLaunch.initializeCurve does not cap antiSnipeSurchargeBps or surchargeToKingPotBps at 10_000** `SteppedTrancheLaunch.initializeCurve` · seen in 2/2 runs · NEW A creator sets `antiSnipeSurchargeBps` or `surchargeToKingPotBps` above `10_000`, so surcharge subtraction underflows in `buy` or `_graduateToV4` and locks buyer funds.	**FIXED**	antiSnipeSurchargeBps>=10_000 and surchargeToKingPotBps>10_000 rejected (SteppedTrancheLaunch.sol:70-71; LaunchpadFactory.sol:122-123).
S1-F27	95	**SteppedTrancheLaunch.sell caps redemptions at basePriceQ96 and locks higher-tranche quote deposits in stalled launches** `SteppedTrancheLaunch.sell` · seen in 1/2 runs · NEW A buyer in a higher tranche who calls `sell` receives only `basePriceQ96` per token, so the contract locks the step price premium.	**PARTIAL**	sell still pays min(basePriceQ96 redemption, pro-rata) (Stepped:193-196). Step premium is mostly unreachable because of the one-tranche target cap (Factory:132-143). → TODO-26
S1-F28	95	**SwapRewardsBlock._accrueSwapRewards uses raw unspecified currency units and inflates points on token-side swaps** `SwapRewardsBlock._accrueSwapRewards` · seen in 1/2 runs · NEW A trader who chooses an exact-input buy or exact-output sell earns orders of magnitude more reward points than an equal quote-denominated swap.	**FIXED**	HookRigOrchestrator.sol:409-412 netRewardVolume is always in project-token units: net unspecified when that leg is the project token, otherwise settledSpecifiedAmount.
S1-F29	95	**submitTWAMMOrder collects no tokens and executePending never subtracts expired order sell rates** `TWAMMBlock.submitTWAMMOrder` · seen in 2/2 runs · NEW Any caller can call `submitTWAMMOrder` with zero token payment, so an attacker sets `sellRate0For1` to `type(uint128).max` for free and makes future orders fail.	**OBSOLETE**	submitTWAMMOrder always reverts TWAMMOrdersDisabled (TWAMMBlock.sol:270-272). Also submitOrder now collects tokens (L80-88); executePending subtracts expired rates (L193-198).
S1-F30	95	**enforceSellCap adds tokensSold to balanceOf(seller) before PoolManager settles input tokens** `VestingSellCapBlock.enforceSellCap` · seen in 2/2 runs · NEW `enforceSellCap` adds `tokensSold` to `balanceOf(seller)` before the router settles the input tokens, which counts `tokensSold` twice and lets a seller exceed the daily cap.	**FIXED**	VestingSellCapBlock.sol:93-101 _preSellBalance nets the locker's currencyDelta and adds tokensSold only when bal<tokensSold for trusted routers, so there is no double count.

ID	Conf.	Finding	Status	Current evidence
S1-F31	95	**VirtualXYKLaunch.buy caps tokensOut at remainingCurveTokens without reducing quoteUsed** `VirtualXYKLaunch.buy` · seen in 2/2 runs · NEW `buy` caps `tokensOut` at the remaining token allocation without lowering `quoteUsed`, so the last buyer pays the full quote amount for fewer tokens.	**FIXED**	When tokensOut is capped, newVirtualQuote and quoteUsed are recomputed from the capped tokensOut (VirtualXYKLaunch.sol:102-107).
S1-F32	95	**_graduateToV4 overflows Math.mulDiv when tokensToSeed / quoteToSeed reaches 2^64** `VirtualXYKLaunch._graduateToV4` · seen in 2/2 runs · NEW `_graduateToV4` overflows `Math.mulDiv` when `tokensToSeed / quoteToSeed` reaches `2^64` on low-decimal quote tokens, so graduation reverts.	**FIXED**	_graduateToV4 uses tiered X192/X160/X128 ratios by tokensToSeed size (VirtualXYKLaunch.sol:201-210), so mulDiv cannot overflow at a 2^64 ratio.
S1-F33	95	**VirtualXYKLaunch.sell increases virtualTokenReserve above invariantK when quoteOut clamps at realQuoteRaised** `VirtualXYKLaunch.sell` · seen in 1/2 runs · NEW A seller whose payout hits the `realQuoteRaised` cap inflates `virtualTokenReserve` above `invariantK` and buys back the excess tokens for one wei.	**FIXED**	sell no longer clamps quoteOut to realQuoteRaised. quoteOut comes from reserves and realQuoteRaised -= quoteOut uses checked math (VirtualXYKLaunch.sol:163-172).
S1-F34	95	**enforceAfterBuy reads token.balanceOf(recipient) before PoolManager settles swap outputs** `WalletCapBlock._enforceWalletCapAfterBuy` · seen in 2/2 runs · NEW `enforceAfterBuy` reads `token.balanceOf` before `PoolManager` settles tokens, so a buyer who batches swaps in one `unlockCallback` can buy more tokens than `maxAllowed`.	**FIXED**	WalletCapBlock.sol:64-71 uses max(balanceOf, transient expected balance) plus bought, then caches postBalance, so batched buys in one unlock add up.
S1-F35	90	**AutoBurnLogic.execute calls ERC20Burnable.burn without fallback and bricks exact-input buys on standard ERC-20 tokens** `AutoBurnBlock._executeAutoBurn` · seen in 1/2 runs · NEW `AutoBurnLogic.execute` calls `ERC20Burnable.burn` with no fallback, so enabling `useTokenBurnCall` on a standard ERC-20 token makes every exact-input buy revert.	**FIXED**	AutoBurnBlock.sol:46-49 wraps ERC20Burnable.burn in try/catch and falls back to safeTransfer(DEAD_ADDRESS).
S1-F36	85	**HoodiSwapRouter.swap retains excess msg.value on exact-output ETH swaps and leaves it drainable by the next caller** `HoodiSwapRouter.swap` · seen in 1/2 runs · NEW `HoodiSwapRouter.swap` retains any excess `msg.value` above `-delta.amount0()`, so an attacker can drain the leftover ETH in a later swap.	**FIXED**	swap computes ethBefore = balance - msg.value and refunds balance - ethBefore after unlock (script/HoodiIntegration.s.sol, HoodiSwapRouter.swap).

6. Scan 2 — 2026-10-15 03:48 (651b515)

3-pass scan of commit 651b515 (SA2-01 – SA2-68). Remediation: b32641f, ede9c74 (second agent audit pass). 66 findings and 2 leads; current status: 36 fixed, 5 obsolete, 4 by-design, 10 partial, 13 present.

ID	Conf.	Finding	Status	Current evidence
S2-F01	98	**Exact-output buy burns quote currency and skips jackpot fee** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` · seen in 3/3 runs · NEW An exact-output buy sets unspecifiedCurrency to quoteCurrency, so _executeAfterSwapDeltaWaterfall sends the buyer quote tokens to DEAD_ADDRESS and skips the KingOfThePool fee.	FIXED	An exact-output buy with BIT_AUTO_BURN or BIT_KING_OF_THE_POOL reverts ExactOutputBuyNotSupported. Burn and King also require unspecifiedCurrency == projectCurrency.
S2-F02	98	**Registration budget check omits ArbRecapture and blocks tax reductions** `AfterSwapDeltaAggregator._validateAggregateReturnDeltaBudget` · seen in 3/3 runs · NEW _validateAggregateReturnDeltaBudget omits the 1000 bps ArbRecapture allowance that setBuySellTax enforces, so ArbRecapture starves later reward blocks and blocks tax reductions.	BY-DESIGN	validateTaxBudget is shared by register and setBuySellTax (taxes only decrease), so cuts are never blocked. Excluding ArbRecapture is documented in docs/06:57.
S2-F03	98	**Start-of-block reference tick never updates after an intra-block swap** `ArbRecaptureBlock.computeCutBps` · seen in 3/3 runs · KNOWN (2 scans) ArbRecaptureLogic.computeCutBps overwrites referenceTick at block start and never advances it within the block, so later traders in that block pay the arbitrage cut.	PARTIAL	The reference now advances to postSwapTick after a charged swap. Sub-threshold moves by earlier traders still add to a later trader's cut, which is the intended anti-split behaviour. → TODO-28
S2-F04	98	**Floor tick comparison is inverted and zero minBuybackReserve locks fees** `AutoBuybackBlock.accumulateAndCheckTrigger` · seen in 3/3 runs · NEW AutoBuybackLogic.accumulateAndCheckTrigger inverts the floor tick check and requires minBuybackReserve above zero, so reserves deploy when price rises or stay locked forever.	PARTIAL	The zero-reserve lock is fixed in init (AutoBuybackBlock.sol:51). The floor-tick comparison at lines 82-83 is still inverted for both token orders. → TODO-22
S2-F05	98	**Window rollover sets anchor to post-swap tick before deviation check** `CircuitBreakerBlock.checkAndAdvance` · seen in 3/3 runs · KNOWN (2 scans) CircuitBreakerLogic.checkAndAdvance sets anchorTick to postSwapTick before the deviation check on window rollover, so an attacker can move the price without limit.	FIXED	On rollover the anchor is set from preSwapTick (clamped) and the deviation check runs against postSwapTick, so the swap's own move is checked.
S2-F06	98	**Same-block swaps compound volatility without updating the observed tick** `DynamicFeeBlock.compute` · seen in 3/3 runs · KNOWN (2 scans) DynamicFeeLogic.compute leaves lastObservedTick unchanged on same-block swaps, so an attacker who sends repeated one-wei swaps in one block pushes the fee to maxFee.	FIXED	Same-block updates happen only if absDelta > rolling and converge to about absDelta+3. Zero-move one-wei swaps cannot push the fee to maxFee.
S2-F07	98	**Pool admin takes all quote surcharges meant for the jackpot** `HookRigOrchestrator.seedKingPotFromLaunchCurve` · seen in 3/3 runs · KNOWN (2 scans) SteppedTrancheLaunch sends currency0 to seedKingPotFromLaunchCurve while the check requires projectCurrency, so the contract credits all buyer surcharges to poolAdmin.	FIXED	seedPotFromLaunchCurve no longer requires projectCurrency; it accepts currency0 as potCurrency (KingOfThePoolBlock.sol:264-271); SteppedTranche mints the matching ERC6909 backing.

ID	Conf.	Finding	Status	Current evidence
S2-F08	98	**One-wei liquidity deposit unlocks same-block JIT swaps forever after cooldown** `JITGuardBlock.recordAddition` · seen in 3/3 runs · KNOWN (2 scans) JITGuardLogic.recordAddition only writes lpAddedBlock when the slot is zero, so an attacker who waits once can add and remove liquidity in one block.	**FIXED**	recordAddition now writes lpAddedBlock = block.number on every addition (JITGuardBlock.sol:27,30), not only when the slot is zero.
S2-F09	98	**Zero post-graduation maxWalletBps reverts when curve wallet cap is set** `LaunchpadFactory._validateWalletCapAlignment` · seen in 3/3 runs · NEW LaunchpadFactory._validateWalletCapAlignment treats a zero post-graduation maxWalletBps as zero percent instead of unlimited, so launches with only a curve cap revert.	**PARTIAL**	Zero maxWalletBps is now consistently normalized to 200 and written back (Factory:219-226). A curve cap above 200 with pool cap 0 still reverts. → TODO-26
S2-F10	98	**Factory caps graduationQuoteTarget at one tranche and blocks upper tranches** `LaunchpadFactory.launchSteppedTranche` · seen in 3/3 runs · NEW LaunchpadFactory.launchSteppedTranche caps graduationQuoteTarget at the quote value of a single tranche, so multi-tranche launches always graduate inside the first tranche.	**PRESENT**	launchSteppedTranche still requires graduationQuoteTarget <= maxTargetForTranche, one tranche at base price (LaunchpadFactory.sol:132-143). → TODO-26
S2-F11	98	**Tick alignment fills uncrossed buckets and dust orders lock ticks forever** `LimitOrdersBlock.processCrossedTicks` · seen in 3/3 runs · KNOWN (2 scans) LimitOrdersLogic.processCrossedTicks aligns prevTick outside the swap range and placeOrder rejects non-empty filled buckets, so an unclaimed one-wei order blocks that tick forever.	**FIXED**	Alignment now stays inside swap range (L348-349,360-361); placeOrder unfreezes filled bucket once price is back on the unfilled side (L179-190).
S2-F12	98	**Oracle weights past elapsed time by new pre-swap tick instead of lastTick** `OracleBlock.updateCheckpoint` · seen in 3/3 runs · KNOWN (2 scans) OracleLogic.updateCheckpoint multiplies the new pre-swap tick across the past elapsed interval and never runs in _afterSwap, so a cross-block swap skews the TWAP.	**FIXED**	Oracle now weights each interval by the pre-swap tick that held during it (v3 semantics), OracleBlock.sol:84-86; regression test test_Fix_OracleWeightsIntervalByItsOwnTick.
S2-F13	98	**Bulk buy skips higher tranche prices and cap rounding blocks graduation** `SteppedTrancheLaunch.buy` · seen in 3/3 runs · KNOWN (2 scans) A buyer who buys across tranches pays one tranche price, and floor division near the allocation cap sets tokensOut to zero.	**PARTIAL**	Cap rounding fixed (Ceil recompute at Stepped:134-137, tokensOut=remainingTokens>0). Bulk buy still uses a single tranche price (Stepped:128-131). → TODO-26
S2-F14	98	**Early tranche buyer sells at average price and takes later deposits** `SteppedTrancheLaunch.sell` · seen in 3/3 runs · KNOWN (2 scans) SteppedTrancheLaunch.sell redeems tokens at the average pool price instead of the stepped curve price, so early tranche buyers take quote deposits from later buyers.	**FIXED**	sell pays min(base-price redemption, pro-rata realQuoteRaised) (SteppedTrancheLaunch.sol:193-196), not the average price, so later deposits cannot be taken.
S2-F15	98	**Raw unspecified currency units inflate points on token-output swaps** `SwapRewardsBlock.accrue` · seen in 3/3 runs · KNOWN (2 scans) SwapRewardsLogic.accrue calculates points from raw unspecifiedCurrency units divided by 1e18, so project-token outputs earn millions more points than quote-output swaps.	**FIXED**	HookRigOrchestrator.sol:409-412 normalizes reward volume to the project token, so raw quote-vs-token unit inflation no longer applies.

ID	Conf.	Finding	Status	Current evidence
S2-F16	98	**Streamed TWAMM deposits never swap and have no claim path** `TWAMMBlock.submitTWAMMOrder` · seen in 3/3 runs · KNOWN (2 scans) TWAMMBlock collects input tokens on submitTWAMMOrder but never swaps them and provides no claim function, so the contract locks streamed tokens forever.	OBSOLETE	submitTWAMMOrder reverts (TWAMMBlock.sol:270-272); no deposits possible. Existing orders refund full totalAmount on cancel (L149-153).
S2-F17	98	**Spot balance snapshot lets sellers bypass cap and freezes small balances** `VestingSellCapBlock.enforceSellCap` · seen in 3/3 runs · KNOWN (2 scans) VestingSellCapLogic.enforceSellCap snapshots spot balanceOf per wallet and floors maxAllowed to zero, so wallet transfers bypass the cap and small holders cannot sell.	PARTIAL	Zero floor fixed (VestingSellCapBlock.sol:74-76 maxAllowed=1) and outflows lower the snapshot (:63-67). Splitting across wallets still bypasses it, documented in docs/04:122. → TODO-21
S2-F18	98	**Graduation truncates sqrtPrice via 32-bit shift and locks excess tokens** `VirtualXYKLaunch._graduateToV4` · seen in 3/3 runs · NEW VirtualXYKLaunch._graduateToV4 shifts Math.sqrt(ratioX128) left by 32 bits when tokensToSeed exceeds uint64.max and donates unsold tokens to an uncollectable LP fee account.	FIXED	An added uint96 branch with 16-bit shift (VirtualXYKLaunch.sol:204-210) leaves negligible precision loss. Donation of leftovers is accepted by design (docs/04:91).
S2-F19	98	**Upward reserve rounding and zero-output buys overcharge buyers on curve** `VirtualXYKLaunch.buy` · seen in 3/3 runs · KNOWN (2 scans) VirtualXYKLaunch.buy charges the full quoteUsed after upward division truncates tokensOut and allows zero-output buys when minTokensOut is zero, so sellers extract the excess quote.	PRESENT	buy allows tokensOut==0 (VirtualXYKLaunch.sol:99) while still charging quoteUsed when minTokensOut==0. There is no zero-output revert. → TODO-26
S2-F20	97	**Merkle verifier skips first 32 bytes and rejects single-leaf proofs** `AccessControlBlock.verifyAccess` · seen in 3/3 runs · KNOWN (2 scans) AccessControlLogic.verifyAccess requires hookData.length >= 32 and skips the first 32 bytes, so valid Merkle proofs from IMsgSender routers and single-leaf trees revert.	FIXED	verifyAccess has a first loop over the whole hookData from offset 0 (len>=32) plus a 32-byte-offset loop; leaf==root covers single-leaf trees.
S2-F21	97	**Fallback raw ERC20.transfer reverts on non-standard tokens and burns quote** `AutoBurnBlock.execute` · seen in 3/3 runs · KNOWN (2 scans) AutoBurnLogic.execute calls raw IERC20.transfer in its catch block and receives quoteCurrency on exact-output buys, so non-standard tokens revert and exact-output buys burn quote assets.	FIXED	safeTransfer fallback (AutoBurnBlock.sol:48). Exact-output buys revert ExactOutputBuyNotSupported (AfterSwapDeltaAggregator.sol:65), and burns run only when unspecified==project.
S2-F22	97	**Router refunds entire ETH balance on any positive msg.value without lock** `HoodiSwapRouter.swap` · seen in 3/3 runs · KNOWN (2 scans) HoodiSwapRouter.swap has no reentrancy guard and refunds address(this).balance whenever msg.value is positive, so a caller who sends one wei takes all router ETH.	FIXED	The refund is limited to `address(this).balance - ethBefore` and is not the whole balance. Pre-existing router ETH cannot be taken with one wei.
S2-F23	97	**Trusted router without IMsgSender lets callers spoof any user via hookData** `HookRigBase._resolveUser` · seen in 3/3 runs · NEW HookRigBase._resolveUser falls back to decoding hookData[:32] whenever a trusted router lacks IMsgSender, so an attacker can spoof any victim address.	FIXED	A trusted router without IMsgSender falls into the catch branch, which returns routerSender, so there is no hookData fallback (AccessControlBlock.sol:242-246).

ID	Conf.	Finding	Status	Current evidence
S2-F24	97	**Attacker can front-run registerPoolStack to seize poolAdmin or block launch** `HookRigOrchestrator.registerPoolStack` · seen in 3/3 runs · KNOWN (2 scans) HookRigOrchestrator.registerPoolStack allows any caller who sets poolAdmin to msg.sender when authorizedCurve is zero, so an attacker can front-run pool registration.	PARTIAL	Launch blocking fixed (factory-only launchpad, deployed-token check at AccessControlBlock.sol:85,99); standard pools are still first-come, admin defaults to msg.sender (:108). → TODO-23
S2-F25	97	**Creator vesting unlocks tokens before graduation and drains curve quote** `LaunchpadFactory._deployTokenAndVest` · seen in 3/3 runs · NEW LaunchpadFactory._deployTokenAndVest starts linear vesting at deployment timestamp instead of graduation, so the creator can sell vested tokens into the active curve.	PRESENT	VestingWallet start is block.timestamp + 1 hours (LaunchpadFactory.sol:327), not graduation. The curve can still be active when vesting releases tokens. → TODO-26
S2-F26	97	**Claim returns original input tokens because swap-on-claim is disabled** `LimitOrdersBlock.claimLimitOrder` · seen in 3/3 runs · NEW LimitOrdersBlock._swapFilledOrderOnClaim always returns false in production contracts, so claimLimitOrder returns the maker input tokens without executing a swap.	FIXED	_swapFilledOrderOnClaim returns true (LimitOrdersBlock.sol:400-402), not overridden. claimOrder swaps through executor (L294-295).
S2-F27	97	**One-wei swap blocks all opposite trades for the rest of the block** `MEVProtectionBlock.enforceAfterSwap` · seen in 3/3 runs · KNOWN (2 scans) MEVProtectionLogic.enforceAfterSwap reverts any opposite-direction swap in the same block, so an attacker who sends a one-wei swap blocks all opposite trades.	PARTIAL	One-wei swaps fall below dirThreshold and cannot set the direction (MEVProtectionBlock.sol:78); a swap of at least dirThreshold ticks still makes opposite swaps of that size revert. → TODO-28
S2-F28	97	**Cancellation retains streamed tokens and truncation dust with no claim** `TWAMMBlock.cancelTWAMMOrder` · seen in 3/3 runs · NEW TWAMMBlock.cancelTWAMMOrder refunds only the unstreamed portion of totalAmount and has no claim path, so the contract locks all streamed tokens forever.	FIXED	cancelOrder refunds full order.totalAmount (TWAMMBlock.sol:149-153), even after expiry. Submissions are disabled anyway.
S2-F29	97	**Sell swap resets transient wallet balance and bypasses maxWalletBps** `WalletCapBlock.enforceAfterBuy` · seen in 3/3 runs · NEW A trader who sells one wei between buys in one unlock callback resets the transient expected balance to zero and acquires tokens above maxWalletBps.	FIXED	WalletCapBlock.sol:95-100: a sell subtracts from the cached expected balance (or balanceOf if unset) instead of resetting to zero.
S2-F30	96	**Zero bounded cut skips floor defense trigger and King round rollover** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` · seen in 2/3 runs · NEW When earlier cuts fill maxAllowedTotalCut or a small swap rounds the cut to zero, _executeAfterSwapDeltaWaterfall skips _processAutoBuyback and _processKingOfThePoolBuy.	FIXED	processWaterfallBuyback always calls accumulateAndCheckTrigger even when bbCut==0. processWaterfallKing always rolls the round and calls processBuy when potCut==0.
S2-F31	96	**Floor defense donates reserves to LPs instead of executing a buyback swap** `AutoBuybackBlock._processAutoBuyback` · seen in 3/3 runs · NEW When AutoBuybackBlock._processAutoBuyback triggers, the waterfall donates the accumulated reserve to LPs via poolManager.donate instead of swapping to support the token price.	BY-DESIGN	docs/04 line 123 and section 3.7 say reserves move into the queued LP donation. flushPendingDonations (AutoBuybackBlock.sol:138-159) intentionally donates.

ID	Conf.	Finding	Status	Current evidence
S2-F32	96	**Initializer accepts zero minBuybackReserve and locks all buyback fees** `AutoBuybackBlock.init` · seen in 3/3 runs · NEW AutoBuybackLogic.init accepts a zero minBuybackReserve while accumulateAndCheckTrigger requires minBuybackReserve above zero, so the contract locks all collected buyback fees forever.	FIXED	AutoBuybackLogic.init line 51 reverts when `params.buybackFeeBps > 0 && params.minBuybackReserve == 0`.
S2-F33	96	**Preset hooks disable beforeInitialize and allow early pool initialization** `HookPermissionMasks.permissionsForBitmap` · seen in 3/3 runs · KNOWN (2 scans) HookPermissionMasks.permissionsForBitmap sets beforeInitialize to false unless BIT_TWAMM is active, so an attacker can initialize preset launchpad pools early and block graduation.	FIXED	beforeInitialize is still off without TWAMM, but presets enable afterInitialize, and _afterInitialize reverts UnauthorizedPoolInitializer for launchpad pools. Factory deploys and registers atomically.
S2-F34	96	**Standard pools do not reset AntiSnipe, WalletCap, or King clocks on init** `HookRigOrchestrator._afterInitialize` · seen in 2/3 runs · NEW HookRigOrchestrator._afterInitialize does not reset the AntiSnipe, WalletCap, or KingOfThePool timers, so standard pools initialized after registration start with decayed clocks.	FIXED	_afterInitialize calls resetGraduationClocks(...,false), which resets the AntiSnipe/WalletCap/King clocks for non-launchpad pools (AntiSnipeBlock.sol:86-98).
S2-F35	96	**Orchestrator skips checking block bitmap against preset hook permissions** `HookRigOrchestrator._configurePoolStackInternal` · seen in 2/3 runs · NEW HookRigOrchestrator._configurePoolStackInternal does not verify that getHookPermissions supports every enabled block, so enabling a return-delta block on ProTradingPoolPresetHook reverts every swap.	FIXED	registerPoolStack calls _validateBitmapCompatibility -> validateBitmapAndPermissions (AccessControlBlock.sol:41-49), which reverts unless the hook address flags cover permissionsForBitmap(bitmap).
S2-F36	96	**One-wei sell between buys resets transient wallet cap balance to zero** `HookRigOrchestrator._runAfterSwapPipeline` · seen in 3/3 runs · NEW HookRigOrchestrator._runAfterSwapPipeline sets the transient wallet balance to zero on any sell, so a buyer who interleaves one-wei sells in unlock bypasses maxWalletBps.	FIXED	WalletCapBlock.sol:94-100: a sell now lowers the expected balance by `sold` (prevExpected - sold) instead of setting it to zero.
S2-F37	96	**Tax update adds 1000 bps for ArbRecapture and blocks lowering taxes** `HookRigOrchestrator.setBuySellTax` · seen in 3/3 runs · NEW HookRigOrchestrator.setBuySellTax adds 1000 bps for BIT_ARB_RECAPTURE while registerPoolStack omits it, so setBuySellTax reverts when poolAdmin tries to lower taxes.	FIXED	setBuySellTax and registration share BuySellTaxLogic.validateTaxBudget (BuySellTaxBlock.sol:143-169); there is no ArbRecapture 1000-bps addition.
S2-F38	96	**Waterfall skips _processKingOfThePoolBuy on exact-output buy swaps** `KingOfThePoolBlock._processKingOfThePoolBuy` · seen in 3/3 runs · NEW The after-swap waterfall skips _processKingOfThePoolBuy when the unspecified token is the quote token, so a buyer who uses an exact-output swap pays zero fee.	FIXED	AfterSwapDeltaAggregator.sol:61-66 reverts ExactOutputBuyNotSupported for exact-output buys when King or AutoBurn is enabled.
S2-F39	96	**Unbounded minBuyAmountForReset collects jackpot fees without crowning a king** `KingOfThePoolBlock.initKing` · seen in 2/3 runs · NEW KingOfThePoolLogic.initKing places no upper bound on minBuyAmountForReset, so an unreachable threshold collects buy fees while currentKing stays zero forever.	PARTIAL	initKing caps minBuyAmountForReset at MAX_MIN_BUY_AMOUNT_FOR_RESET=1e9 ether (KingOfThePoolBlock.sol:21,69). That can still be unreachable for small-supply tokens. → TODO-28

ID	Conf.	Finding	Status	Current evidence
S2-F40	96	**Exact-output buys skip KingOfThePoolBlock and avoid the jackpot fee** `KingOfThePoolBlock.processBuy` · seen in 3/3 runs · KNOWN (2 scans) AfterSwapDeltaAggregator skips KingOfThePoolLogic.processBuy when unspecifiedCurrency is quoteCurrency, so any buyer who uses an exact-output swap pays zero jackpot fee.	**FIXED**	Exact-output buys revert with ExactOutputBuyNotSupported (AfterSwapDeltaAggregator.sol:61-66), so the jackpot fee can no longer be skipped this way.
S2-F41	96	**Factory transfers full unvested supply to NativeTickLaunch and locks excess** `LaunchpadFactory.launchNativeTick` · seen in 3/3 runs · NEW LaunchpadFactory.launchNativeTick transfers all available tokens to NativeTickLaunch even when the liquidity bands use less, so the unseeded tokens stay locked forever.	**PRESENT**	Factory transfers the full totalSupply-vesting to the curve (Factory:333-334) while bands may use less (Factory:168-174). NativeTickLaunch has no sweep. → TODO-27
S2-F42	96	**Preset hooks passed to launchVirtualXYK lack beforeInitialize protection** `LaunchpadFactory.launchVirtualXYK` · seen in 2/3 runs · KNOWN (2 scans) LaunchpadFactory.launchVirtualXYK accepts preset hooks whose permissions disable beforeInitialize, so an attacker can initialize the pool early and block graduation.	**FIXED**	_resolveHook rejects any hook other than defaultOrchestrator (LaunchpadFactory.sol:192-196), so preset hooks cannot be passed.
S2-F43	96	**Fee-on-transfer limit order cancellation drains shared hook token reserves** `LimitOrdersBlock.placeLimitOrder` · seen in 2/3 runs · NEW LimitOrdersLogic.placeOrder records inputAmount without measuring received tokens, so a maker who cancels a fee-on-transfer order withdraws other users' tokens.	**FIXED**	placeOrder checks balance delta: `token.balanceOf(this) - balBefore != inputAmount` reverts (LimitOrdersBlock.sol:198-200).
S2-F44	96	**Unguarded executeFillSwap lets any caller sweep executor balances** `LimitOrderSwapExecutor.executeFillSwap` · seen in 2/3 runs · NEW LimitOrderSwapExecutor.executeFillSwap has no caller check and sends any remaining token or ETH balance to recipient, so any user can sweep funds held by the executor.	**FIXED**	executeFillSwap: `if (msg.sender != hook) revert OnlyHookCaller` (LimitOrdersBlock.sol:51).
S2-F45	96	**Updating managed fee before pool initialization reverts in PoolManager** `ManagedFeeBlock.setManagedPoolFee` · seen in 2/3 runs · NEW ManagedFeeBlock.setManagedPoolFee calls poolManager.updateDynamicLPFee unconditionally, so the fee manager cannot update the staged fee before the pool initializes.	**PRESENT**	setManagedPoolFee (ManagedFeeBlock.sol:72-76) still calls _updatePoolManagerDynamicFee -> poolManager.updateDynamicLPFee unconditionally, which reverts before initialization. → TODO-29
S2-F46	96	**Unallocated and rounding-leftover curve tokens stay locked in NativeTickLaunch** `NativeTickLaunch.unlockCallback` · seen in 3/3 runs · KNOWN (2 scans) NativeTickLaunch.unlockCallback settles only the band liquidity deltas and has no sweep path, so all unallocated tokens stay locked in the contract forever.	**PRESENT**	unlockCallback skips invalid or zero bands and settles only the band deltas (NativeTickLaunch.sol:72-103). There is no sweep or withdraw for leftover tokens. → TODO-27
S2-F47	96	**Attacker binds referrer to untrusted router and siphons swap fees** `ReferralSplitBlock.registerReferrer` · seen in 2/3 runs · NEW ReferralSplitBlock.registerReferrer permanently sets traderReferrer for msg.sender, so an attacker who calls it through an untrusted router takes 3% of swaps through that router.	**PRESENT**	ReferralSplitBlock.registerReferrer (line 93-95) binds msg.sender with no router check. For untrusted routers ctx.trader is the router, so a router that can make arbitrary calls is exposed. → TODO-24

ID	Conf.	Finding	Status	Current evidence
S2-F48	96	**Stepped curve graduation truncates sqrtPrice via 32-bit shift** `SteppedTrancheLaunch._graduateToV4` · seen in 2/3 runs · NEW SteppedTrancheLaunch._graduateToV4 shifts Math.sqrt(ratioX128) left by 32 bits when tokensToSeed exceeds uint64.max, which truncates the initial pool price.	FIXED	Tiered X192/X160(<<16)/X128(<<32) sqrt computation (SteppedTrancheLaunch.sol:237-246) limits truncation to negligible precision.
S2-F49	96	**Stepped curve graduation donates leftover tokens to locked LP position** `SteppedTrancheLaunch.unlockCallback` · seen in 2/3 runs · NEW SteppedTrancheLaunch.unlockCallback passes leftover project tokens to poolManager.donate on its own locked LP position, so nobody can ever claim or trade them.	BY-DESIGN	Leftover donation to the locked graduation LP is documented as an accepted design limit (docs/04:91; docs/07 TODO-21 "Accepted").
S2-F50	96	**Unbounded loop in executePending reverts swaps when dust orders accumulate** `TWAMMBlock.executePending` · seen in 2/3 runs · NEW An attacker submits many one-year dust orders in one timestamp so the unbounded while loop in TWAMMLogic.executePending runs out of gas on later blocks.	OBSOLETE	No new orders can be submitted (TWAMMBlock.sol:270-272), so the active list cannot grow. Loop is also capped by MAX_ACTIVE_TWAMM_ORDERS=256 (L17,75).
S2-F51	96	**Fee-on-transfer TWAMM order cancellation drains shared hook token reserves** `TWAMMBlock.submitTWAMMOrder` · seen in 2/3 runs · NEW TWAMMLogic.submitOrder records totalAmount without measuring received tokens, so a caller who cancels a fee-on-transfer order in the same block takes other users' tokens.	OBSOLETE	submitTWAMMOrder reverts (TWAMMBlock.sol:270-272). submitOrder also verifies the received balance delta (L85-87).
S2-F52	96	**Unupdated invariantK after rounded buy lets seller drain prior buyer quote** `VirtualXYKLaunch.sell` · seen in 3/3 runs · KNOWN (2 scans) VirtualXYKLaunch leaves invariantK unchanged when upward division on buy increases the reserve product, so a seller extracts prior buyers' excess quote in sell.	PRESENT	invariantK is set only in initializeCurve (VirtualXYKLaunch.sol:78). buy's Ceil rounding and zero-output buys raise the reserve product, which sell pays out. → TODO-26
S2-F53	96	**Graduation donates leftover tokens to an unclaimable LP fee account** `VirtualXYKLaunch.unlockCallback` · seen in 3/3 runs · NEW VirtualXYKLaunch.unlockCallback passes leftover project tokens to poolManager.donate on a locked position with no fee-collection function, so the contract freezes those tokens forever.	BY-DESIGN	unlockCallback donates leftovers to the locked LP (VirtualXYKLaunch.sol:252-264), an accepted limit per docs/04:91 and docs/07 TODO-21.
S2-F54	94	**Reward points use raw unspecified token units without normalization** `SwapRewardsBlock._accrueSwapRewards` · seen in 3/3 runs · NEW SwapRewardsBlock._accrueSwapRewards computes points from the raw unspecified token amount, so a trader who selects the project token as the unspecified leg inflates points.	FIXED	HookRigOrchestrator.sol:409-412 computes points from project-token volume whichever leg is unspecified.
S2-F55	92	**Tax update rejects zero recipient sentinel before checking stored recipient** `BuySellTaxBlock._updateBuySellTax` · seen in 3/3 runs · NEW BuySellTaxLogic.update rejects a zero newRecipient before it checks the stored taxRecipient, so the poolAdmin cannot use zero to preserve the existing recipient.	PRESENT	BuySellTaxBlock.sol:107 still reverts on newRecipient==0 when any tax>0, so the :112 keep-existing branch only works when both taxes are zero. → TODO-29

ID	Conf.	Finding	Status	Current evidence
S2-F56	92	**Preset hooks allow registering extra blocks missing from hook permissions** `FairLaunchPresetHook.getHookPermissions` · seen in 2/3 runs · NEW Preset hook contracts override getHookPermissions with fixed preset flags but accept any Tier 3 bitmap in registerPoolStack, so unsupported return-delta blocks revert every swap.	FIXED	validateBitmapAndPermissions requires the hook address flags to cover permissionsForBitmap(bitmap) (AccessControlBlock.sol:45-48), called in registerPoolStack:100.
S2-F57	92	**Premature graduation call or curve LP seeding corrupts post-graduation timers** `HookRigOrchestrator.notifyPoolGraduated` · seen in 1/3 runs · KNOWN (2 scans) HookRigOrchestrator.notifyPoolGraduated allows launchpadFactory to reset clocks before graduation and records the curve's full-range LP block in JITGuard's shared rangeKey slot.	FIXED	resetGraduationClocks allows one call only (AntiSnipeBlock.sol:82 PoolAlreadyGraduated); curves call it right after initialize; _beforeAddLiquidity skips _recordLPAddition for the curve (Orchestrator:273).
S2-F58	92	**Unbounded anti-snipe surcharge and duration divert buyer deposits to creator** `SteppedTrancheLaunch.initializeCurve` · seen in 1/3 runs · KNOWN (2 scans) SteppedTrancheLaunch.initializeCurve accepts antiSnipeSurchargeBps up to 9999 and unlimited antiSnipeBlocks, so a creator can divert 99.99% of buyer quote payments into creator claims.	PRESENT	Only antiSnipeSurchargeBps<10_000 is enforced (Stepped:70, Factory:122). antiSnipeBlocks is unbounded and surcharges have no buyer refund path. → TODO-26
S2-F59	92	**Unbounded active TWAMM order loop runs out of gas during swaps** `TWAMMBlock._executePendingTWAMMOrders` · seen in 2/3 runs · NEW TWAMMBlock._executePendingTWAMMOrders iterates through every active order on each swap and liquidity call, so an attacker who submits many dust orders blocks the pool.	OBSOLETE	Active order list cannot grow with submissions disabled (TWAMMBlock.sol:270-272). It is also capped at 256 (L75).
S2-F60	85	**Pre-initialization TWAMM order timestamp overwrite inflates pool sell rate forever** `TWAMMBlock._initTWAMM` · seen in 1/3 runs · NEW Submitting a TWAMM order before pool initialization causes _initTWAMM to overwrite lastVirtualOrderTimestamp, so executePending drops the expired order without subtracting sellRatePerSecond.	OBSOLETE	_initTWAMM has no callers; initAfterInitialize sets the timestamp. Pre-init submission impossible (submit disabled); expired rates now subtracted (L193-198).
S2-F61	82	**Zero managed fee cap is replaced with five percent cap** `ManagedFeeBlock.stage` · seen in 1/3 runs · NEW ManagedFeeLogic.stage replaces a zero feeCap with MAX_MANAGED_FEE_CAP, so the fee manager can raise the pool fee to five percent despite a zero cap.	FIXED	ManagedFeeLogic.stage uses params.feeCap as-is (ManagedFeeBlock.sol:32); a zero cap stays zero and is no longer replaced with MAX_MANAGED_FEE_CAP.
S2-F62	80	**Empty or sub-unit native tick bands graduate pool with zero liquidity** `NativeTickLaunch.initializeAndSeedNativeTicks` · seen in 1/3 runs · NEW NativeTickLaunch.initializeAndSeedNativeTicks allows empty bands or bands whose liquidity rounds to zero, which graduates the pool with zero liquidity and locks all tokens.	PARTIAL	Factory rejects empty bands arrays (Factory:163-165), but zero-amount, invalid or zero-liquidity bands are skipped (NativeTick:73-85), so liquidity can be zero. → TODO-27
S2-F63	78	**Standard pools allow any caller to initialize at an extreme price** `HookRigOrchestrator._beforeInitialize` · seen in 1/3 runs · NEW HookRigOrchestrator._beforeInitialize checks the caller only for launchpad pools, so an attacker can initialize a registered standard pool at an extreme tick.	PRESENT	HookRigOrchestrator.sol:203 checks the caller only when meta.isLaunchpadPool; any caller can initialize a registered standard pool at any sqrtPrice. → TODO-23

ID	Conf.	Finding	Status	Current evidence
S2-F64	78	**Swaps through untrusted routers crown the router and lock the jackpot** `HookRigOrchestrator.claimKingPot` · seen in 1/3 runs · NEW Swapping through an untrusted router sets round.winner to the router contract address, so the buyer cannot call claimKingPot and the jackpot stays locked.	**PRESENT**	An untrusted router still resolves to routerSender (AccessControlBlock.sol:250); processBuy crowns it (KingOfThePoolBlock.sol:158); validateAndMarkClaimed requires msg.sender==winner (:222). → TODO-24
S2-F65	78	**Untrusted router crowned as king cannot call validateAndMarkClaimed** `KingOfThePoolBlock.validateAndMarkClaimed` · seen in 1/3 runs · NEW KingOfThePoolLogic.validateAndMarkClaimed requires msg.sender to equal round.winner, so jackpots won through an untrusted router contract can never be claimed.	**PRESENT**	validateAndMarkClaimed line 222 still requires msg.sender==round.winner. resolveUser returns the untrusted router itself (AccessControlBlock.sol:250), and nothing prevents crowning it. → TODO-24
S2-F66	75	**Cancelling a fee-on-transfer limit order withdraws full nominal input amount** `LimitOrdersBlock.cancelLimitOrder` · seen in 1/3 runs · NEW LimitOrdersBlock.cancelLimitOrder transfers the unadjusted order.inputAmount from the shared hook balance, so cancelling a fee-on-transfer order drains other makers' escrowed tokens.	**FIXED**	Exact-received check in placeOrder (L198-200) plus per-pool escrow guard InsufficientPoolEscrow in cancelOrder (L240-244).
S2-L01	lead	**JIT exit penalty configuration is stored but never read on removal** `JITGuardBlock._initJITGuard` Code smells: `JITGuardBlock._initJITGuard` validates `jitExitPenaltyBps <= MAX_JIT_PENALTY_BPS` and writes `_jitConfigs[poolId].jitExitPenaltyBps` (lines 1689-1695), but no function in `JITGuardBlock` or `HookRigOrchestrator` ever reads `jitExitPenaltyBps`. — JITGuardBlock stores jitExitPenaltyBps in _jitConfigs during initializ…	**PARTIAL**	jitExitPenaltyBps is now read in enforceOnRemove (JITGuardBlock.sol:44-46), but only to force minHold=1; no penalty is ever deducted, despite docs/04:119. → TODO-16
S2-L02	lead	**Shared native ETH limit order escrow lacks per-pool balance isolation** `LimitOrdersBlock.placeLimitOrder` Code smells: `LimitOrdersLogic.placeOrder` (lines 2123-2129) holds native ETH and ERC-20 limit order escrows in the shared `HookRigOrchestrator` balance without checking `_poolMeta[poolId].configured`. — LimitOrdersBlock stores native ETH escrows in the shared hook balance, though draining native ETH requires another path that u…	**FIXED**	Per-pool escrow `limitOrderPoolEscrow[poolId][currency]` checked on cancel/claim (L240-244,286-290). Native ETH requires msg.value==inputAmount (L194).

7. Scan 3 — 2026-10-15 07:18 (ede9c74)

1-pass scan of commit `ede9c74` (pass-3 findings). Remediation: b0859d2 (feat: audit pass 3 and fuzz testing). 69 findings and 6 leads; current status: 39 fixed, 4 obsolete, 4 by-design, 17 partial, 11 present.

ID	Conf.	Finding	Status	Current evidence
S3-F01	98	**Exact-output buy sends buyer quote currency to the dead address** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` On an exact-output buy, _executeAfterSwapDeltaWaterfall passes quoteCurrency to AutoBurnLogic, so the hook sends quote tokens to the dead address.	**FIXED**	An exact-output buy reverts before the burn. AutoBurn is gated by `ctx.unspecifiedCurrency == ctx.projectCurrency` (AfterSwapDeltaAggregator.sol:113-116).
S3-F02	98	**Preset hooks disable beforeInitialize and let callers block graduation** `HookPermissionMasks.permissionsForBitmap` permissionsForBitmap disables beforeInitialize when BIT_TWAMM is off, so an attacker can initialize a preset launchpad pool early and block graduation forever.	**FIXED**	_afterInitialize (enabled whenever bitmap!=0) reverts if sender isn't authorizedCurve/launchpadFactory (HookRigOrchestrator.sol:221-223), blocking early init.
S3-F03	98	**Quote currency surcharge seed is diverted to poolAdmin in seedKingPotFromLaunchCurve** `KingOfThePoolBlock.seedKingPotFromLaunchCurve` KingOfThePoolLogic.seedPotFromLaunchCurve requires the project token while SteppedTrancheLaunch sends the quote token, so the pool admin takes the Round 1 pot.	**FIXED**	seedPotFromLaunchCurve (KingOfThePoolBlock.sol:264-271) accepts any currency into an empty pot and reverts otherwise. No poolAdmin credit path remains.
S3-F04	98	**Currency check in seedPotFromLaunchCurve sends launchpad jackpot seed to poolAdmin** `KingOfThePoolBlock.seedPotFromLaunchCurve` The seedPotFromLaunchCurve function requires projectCurrency while SteppedTrancheLaunch sends currency0, so the contract sends the Round 1 jackpot surcharge to poolAdmin.	**FIXED**	The quote currency0 seed from SteppedTrancheLaunch.sol:260 is accepted when the pot is empty (KingOfThePoolBlock.sol:266-271). There is no poolAdmin diversion.
S3-F05	98	**KingOfThePoolLogic.seedPotFromLaunchCurve rejects quoteCurrency and credits poolAdmin** `KingOfThePoolLogic.seedPotFromLaunchCurve` The poolAdmin receives all graduation surcharge funds from SteppedTrancheLaunch because seedPotFromLaunchCurve requires projectCurrency while _graduateToV4 passes quoteCurrency.	**FIXED**	seedPotFromLaunchCurve no longer requires projectCurrency (only empty pot or matching currency) and reverts KingPotSeedNotAccepted instead of crediting poolAdmin.
S3-F06	98	**SteppedTrancheLaunch._graduateToV4 passes currency0 to seedKingPotFromLaunchCurve and diverts pot** `SteppedTrancheLaunch._graduateToV4` SteppedTrancheLaunch._graduateToV4 passes poolKey.currency0 to seedKingPotFromLaunchCurve, which checks currency == meta.projectCurrency, so the contract sends the jackpot seed to poolAdmin.	**FIXED**	seedPotFromLaunchCurve now accepts the curve's currency0 and credits a live pot, else reverts. It never credits poolAdmin (KingOfThePoolBlock.sol:264-271).
S3-F07	98	**TWAMM order cancellation withholds streamed tokens with no claim path** `TWAMMBlock.cancelTWAMMOrder` A user who submits a TWAMM order loses all streamed tokens forever, because TWAMMBlock deducts streamed amounts on cancel and has no claim function.	**FIXED**	cancelOrder refunds full totalAmount, no streamed deduction (TWAMMBlock.sol:149-153). Submission is disabled.
S3-F08	98	**TWAMMLogic.executePending retires orders without a swap or claim credit** `TWAMMBlock.executePending` TWAMMBlock records virtual volume and deletes expired orders without a swap or claim mechanism, so the contract locks user tokens forever.	**FIXED**	Retired/expired orders stay cancellable for a full totalAmount refund (TWAMMBlock.sol:126-153). New orders are disabled (L270-272).

ID	Conf.	Finding	Status	Current evidence
S3-F09	98	**Streamed TWAMM order deposits are never swapped and stay locked forever** `TWAMMBlock.submitTWAMMOrder` The contract holds tokens from submitTWAMMOrder and never swaps or pays out the streamed share, so every order loses its streamed tokens forever.	OBSOLETE	submitTWAMMOrder reverts TWAMMOrdersDisabled (TWAMMBlock.sol:270-272). Any legacy order refunds in full on cancel.
S3-F10	95	**Exact-output buy skips the KingOfThePool fee check** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` AfterSwapDeltaAggregator skips processWaterfallKing when unspecifiedCurrency is the quote token, so a buyer who uses an exact-output swap pays zero pot fee.	FIXED	An exact-output buy on a King-enabled pool reverts ExactOutputBuyNotSupported, so the pot fee cannot be skipped (AfterSwapDeltaAggregator.sol:60-65).
S3-F11	95	**Zero rounded arbitrage cut still advances referenceTick to postSwapTick** `ArbRecaptureBlock._computeArbRecaptureCutBps` ArbRecaptureLogic.computeCutBps updates referenceTick to postSwapTick even when rawCutBps rounds to zero, so an arbitrageur splits a swap and pays zero fee.	FIXED	computeCutBps only sets `state.referenceTick = postSwapTick` when `rawCutBps > 0` (ArbRecaptureBlock.sol:69-71).
S3-F12	95	**Split swaps reset referenceTick in computeCutBps with zero fee** `ArbRecaptureBlock.computeCutBps` ArbRecaptureLogic.computeCutBps updates referenceTick when rawCutBps rounds to zero, so a trader splits one large swap into small steps and pays zero fee.	FIXED	referenceTick advances only when rawCutBps>0, and the tick move is measured from referenceTick, so split swaps add up their moves.
S3-F13	95	**One-excess-tick swaps advance referenceTick in ArbRecaptureLogic with zero cut** `ArbRecaptureLogic.computeCutBps` An arbitrageur can split a swap into steps of arbTriggerTickDelta plus one tick to reset referenceTick and pay zero recapture fee.	FIXED	A zero-cut swap no longer advances referenceTick (`if (rawCutBps > 0)`), and refTickMoved adds up the steps since the last charged swap.
S3-F14	95	**Inverted tick check triggers buyback above the floor price** `AutoBuybackBlock._processAutoBuyback` accumulateAndCheckTrigger inverts the tick comparison for both token orders, so the contract deploys buyback reserves only above the floor price.	PRESENT	AutoBuybackBlock.sol:82-83 is unchanged since commit 651b515. In test_Block22 (project=currency1), a zeroForOne buy pushing the tick down triggers the "floor". → TODO-22
S3-F15	95	**Shared router key in _enforceJITGuardOnRemove lets attackers block LP withdrawals** `JITGuardBlock._enforceJITGuardOnRemove` JITGuardLogic.recordAddition writes to the shared router key, so an attacker who adds one wei of liquidity blocks withdrawals for the position owner.	PARTIAL	recordAddition (JITGuardBlock.sol:24-26) adds lpOwner to the key only when salt==0 on a resolved router; untrusted routers and salt!=0 keys stay shared (router,ticks,salt). → TODO-16
S3-F16	95	**Shared router and salt overwrite lpAddedBlock in _recordLiquidityAddition** `JITGuardBlock._recordLiquidityAddition` recordAddition overwrites lpAddedBlock for any shared sender and salt, so an attacker can add one wei of liquidity to reset a victim cooldown.	PARTIAL	A shared sender+salt key is still overwritten by any 1-wei add (JITGuardBlock.sol:26-27); fixed only for the trusted-router salt==0 case (lpOwner in key). → TODO-16
S3-F17	95	**Router senderKey write in _recordLPAddition locks other router LPs** `JITGuardBlock._recordLPAddition` JITGuardLogic.recordAddition overwrites the shared senderKey block timestamp, so an attacker who adds one wei via the same router locks another LP position.	PARTIAL	senderKey is per-owner only for salt==0 on a trusted router (JITGuardBlock.sol:24-26); untrusted-router/salt!=0 adders still overwrite the shared key and block withdrawals for up to 7200 blocks. → TODO-16

ID	Conf.	Finding	Status	Current evidence
S3-F18	95	**Router cooldown lookup in enforceOnRemove blocks LP withdrawals** `JITGuardBlock.enforceOnRemove` The recordAddition function stores the cooldown under the router address, so an attacker who adds one wei of liquidity blocks withdrawals for other users.	PARTIAL	enforceOnRemove reads the same shared (sender,ticks,salt) key for untrusted routers or salt!=0 (JITGuardBlock.sol:49-52); per-owner key only for trusted router with salt 0. → TODO-16
S3-F19	95	**One-wei deposit via a shared router resets all LP hold timers** `JITGuardBlock.recordAddition` JITGuardLogic.recordAddition overwrites the shared router timestamp on every deposit, so any depositor resets the withdrawal hold period for other router users.	PARTIAL	recordAddition still writes the shared router key for untrusted routers/nonzero salt (JITGuardBlock.sol:26-27); trusted salt-0 positions are keyed per owner. → TODO-16
S3-F20	95	**Shared senderKey in JITGuardLogic.recordAddition lets attackers freeze LP removals** `JITGuardLogic.recordAddition` An attacker who adds one wei of liquidity through a shared router resets senderKey and blocks liquidity removal for other users.	PARTIAL	senderKey includes lpOwner only when salt==0 and lpOwner!=sender (JITGuardBlock.sol:24-26); a shared untrusted-router key can still be reset by 1-wei adds. → TODO-16
S3-F21	95	**Creator vesting starts before graduation and drains buyer quote deposits** `LaunchpadFactory._deployTokenAndVest` LaunchpadFactory._deployTokenAndVest unlocks creator tokens before graduation, so the creator can sell those tokens into the curve and take buyer quote tokens.	PRESENT	Vesting starts at deployment+1h (LaunchpadFactory.sol:327), not at graduation. Creator can sell released tokens into the active curve. → TODO-26
S3-F22	95	**Single-tranche graduation cap forces SteppedTrancheLaunch to graduate in tranche zero** `LaunchpadFactory.launchSteppedTranche` LaunchpadFactory caps graduationQuoteTarget at the quote value of one tranche, so SteppedTrancheLaunch always graduates in tranche zero and skips higher tranches.	PRESENT	graduationQuoteTarget is capped at maxTargetForTranche, one tranche at base price (LaunchpadFactory.sol:132-143). → TODO-26
S3-F23	95	**Hardcoded false in _swapFilledOrderOnClaim refunds unswapped input tokens** `LimitOrdersBlock._swapFilledOrderOnClaim` LimitOrdersBlock._swapFilledOrderOnClaim returns false in production hooks, so claimLimitOrder returns the original input token without a swap.	FIXED	_swapFilledOrderOnClaim returns true (LimitOrdersBlock.sol:400-402); no override in HookRigOrchestrator.
S3-F24	95	**Filled limit orders never swap and refund original input tokens on claim** `LimitOrdersBlock.claimLimitOrder` LimitOrdersBlock._swapFilledOrderOnClaim returns false in HookRigOrchestrator, so LimitOrdersLogic.claimOrder returns the original input tokens to the maker instead of output tokens.	FIXED	claimOrder calls _executeClaimSwap via executor when swapOnClaim, paying output tokens (LimitOrdersBlock.sol:294-295,319-327).
S3-F25	95	**One-wei unclaimed order in a crossed bucket blocks placeLimitOrder forever** `LimitOrdersBlock.placeLimitOrder` An attacker who places a one-wei order and never claims it after fill keeps bucket.totalInputAmount above zero, which blocks placeLimitOrder at that tick forever.	FIXED	placeOrder resets bucket.filled when price is back on the unfilled side (LimitOrdersBlock.sol:179-190). Reverts only while target is still crossed.
S3-F26	95	**Unclaimed dust in a crossed bucket makes placeOrder revert forever** `LimitOrdersBlock.placeOrder` An attacker leaves a tiny unclaimed order in a crossed bucket, and placeOrder rejects all new orders at that tick forever.	FIXED	Same placeOrder reset (LimitOrdersBlock.sol:183-188). Dust only blocks while price stays past target, when a new order would be marketable anyway.

ID	Conf.	Finding	Status	Current evidence
S3-F27	95	**Unclaimed one-wei order locks LimitOrdersLogic.placeOrder at that tick forever** `LimitOrdersLogic.placeOrder` An attacker leaves a one-wei filled order unclaimed at a tick, so LimitOrdersLogic.placeOrder reverts with BucketAlreadyFilled for every caller at that tick forever.	**FIXED**	BucketAlreadyFilled is now raised only if currentTick is still past targetTick; otherwise filled is cleared (LimitOrdersBlock.sol:183-188).
S3-F28	95	**Floor rounding at the allocation cap in SteppedTrancheLaunch.buy blocks graduation** `SteppedTrancheLaunch.buy` SteppedTrancheLaunch.buy rounds netQuoteForTokens down when a buy exceeds remainingTokens, so the last dust balance returns zero tokens and blocks graduation for every buyer.	**FIXED**	In the cap branch, tokensOut=remainingTokens and netQuoteForTokens is recomputed with Math.Rounding.Ceil (SteppedTrancheLaunch.sol:132-137).
S3-F29	95	**Single marginal price lookup in SteppedTrancheLaunch.buy skips higher tranche steps** `SteppedTrancheLaunch.buy` SteppedTrancheLaunch.buy prices the whole order at currentMarginalPriceQ96, so a buyer in tranche zero buys the entire allocation at basePriceQ96 and skips all price steps.	**PARTIAL**	Whole order is still priced at currentMarginalPriceQ96 (Stepped:128-131). The one-tranche target cap (Factory:132-143) stops buying the entire allocation. → TODO-26
S3-F30	95	**Live balanceOf snapshot in _enforceVestingSellCap allows flash-loan sell cap bypass** `VestingSellCapBlock._enforceVestingSellCap` enforceSellCap reads the live balance of the seller during _afterSwap, so a seller can borrow pool tokens before the swap and sell their full balance.	**PARTIAL**	PoolManager take is netted via currencyDelta (VestingSellCapBlock.sol:97). External flash loans still raise balanceOf and the first-of-epoch snapshot (:60-62). → TODO-28
S3-F31	95	**Pool token borrow before swap inflates startOfEpochBalance in enforceSellCap** `VestingSellCapBlock.enforceSellCap` A seller who flash-takes projectCurrency from poolManager before the first daily swap inflates ep.startOfEpochBalance and sells their entire balance in one transaction.	**FIXED**	_preSellBalance adds TransientStateLibrary.currencyDelta for the locker (VestingSellCapBlock.sol:96-97), so a flash take from poolManager is subtracted.
S3-F32	95	**Zero post-swap seller balance in enforceSellCap reverts every sell above one wei** `VestingSellCapBlock.enforceSellCap` When currentBal is zero, VestingSellCapLogic.enforceSellCap sets ep.startOfEpochBalance to tokensSold, so newSoldTotal > maxAllowed reverts every sell above one wei.	**FIXED**	VestingSellCapBlock.sol:56 `if (preSellBal == 0) return;` skips the cap instead of setting startOfEpochBalance=tokensSold.
S3-F33	95	**Setting startOfEpochBalance to tokensSold in VestingSellCapLogic.enforceSellCap reverts sells** `VestingSellCapLogic.enforceSellCap` Every sell through an untrusted router reverts with DailyVestingSellCapExceeded because enforceSellCap sets startOfEpochBalance to tokensSold and requires a smaller sale.	**PARTIAL**	Zero-balance routers are skipped (:56). An untrusted router that pre-settles or holds the input still gets preSellBal==tokensSold (:97), so maxAllowed<tokensSold and it reverts. → TODO-24
S3-F34	95	**Untrusted router contract bypasses maxWalletBps in _enforceWalletCapAfterSwap** `WalletCapBlock._enforceWalletCapAfterSwap` WalletCapLogic.enforceAfterBuy checks the token balance of an untrusted router instead of the recipient, so a buyer who uses a custom router bypasses maxWalletBps.	**PRESENT**	resolveUser returns routerSender for untrusted routers (AccessControlBlock.sol:238-249). enforceAfterBuy checks balanceOf(router) (WalletCapBlock.sol:64), and no router guard exists. → TODO-24
S3-F35	95	**Untrusted router contract bypasses WalletCapBlock.enforceAfterSwap** `WalletCapBlock.enforceAfterSwap` The enforceAfterBuy function checks the token balance of trader, so a buyer who uses an untrusted router contract can buy the entire token supply.	**PRESENT**	Same path: Orchestrator.sol:388-397 passes ctx.trader (the untrusted router) to the wallet cap, which checks the router's balance, not the recipient's. → TODO-24

ID	Conf.	Finding	Status	Current evidence
S3-F36	95	**Zero router token balance in WalletCapLogic.enforceAfterBuy lets buyers exceed maxWalletBps** `WalletCapLogic.enforceAfterBuy` A buyer who swaps through an untrusted router contract makes AccessControlLogic.resolveUser return the router address, so WalletCapLogic.enforceAfterBuy checks the zero router balance.	**PRESENT**	The router's balance (often zero) plus netBought is compared to maxAllowed (WalletCapBlock.sol:64-69). Repeated swaps through a custom router get around maxWalletBps. → TODO-24
S3-F37	90	**HookData fallback in _resolveUser lets callers spoof allowlisted users** `AccessControlBlock._resolveUser` AccessControlLogic.resolveUser reads the user address from hookData when msgSender fails, so an attacker can impersonate an allowlisted user.	**FIXED**	AccessControlBlock.sol:238 resolveUser ignores hookData (unnamed param); trusted router returns msgSender() or routerSender, untrusted returns routerSender.
S3-F38	90	**Unauthenticated hookData fallback in resolveUser allows user impersonation** `AccessControlBlock.resolveUser` The resolveUser function reads the user address from caller hookData when msgSender reverts, so an attacker can impersonate any allowlisted address.	**FIXED**	resolveUser catch branch returns routerSender, not a hookData-decoded address (AccessControlBlock.sol:242-246).
S3-F39	90	**Trusted router catch block decodes unverified victim address from hookData** `AccessControlLogic.resolveUser` An attacker passes a victim address in hookData when IMsgSender.msgSender reverts, which sets startOfEpochBalance and lastSwapTimestamp for the victim.	**FIXED**	AccessControlLogic.resolveUser catch block now `return routerSender;`, so no victim address is decoded from hookData.
S3-F40	90	**Missing hook permission check in _validateBitmapCompatibility causes swaps to revert** `HookRigOrchestrator._validateBitmapCompatibility` HookRigOrchestrator.registerPoolStack does not check getHookPermissions against the bitmap, so return-delta blocks on ProTradingPoolPresetHook make every swap revert.	**FIXED**	_validateBitmapCompatibility (Orchestrator:90-92) checks the bitmap against the hook-address permission flags through validateBitmapAndPermissions (AccessControlBlock.sol:45-48).
S3-F41	90	**Anyone can front-run registerPoolStack to hijack poolAdmin or block launch** `HookRigOrchestrator.registerPoolStack` registerPoolStack checks no caller permission when authorizedCurve is zero, so an attacker can front-run token deployment and lock the PoolId configuration forever.	**PARTIAL**	Token-deployment front-run blocked by _isUndeployedToken (AccessControlBlock.sol:99); a standard pool of existing tokens can still be registered first, with admin=msg.sender (:108). → TODO-23
S3-F42	90	**Preset hooks accept bitmaps that exceed their static hook permissions** `HookRigPresets.registerPoolStack` Preset hook contracts do not verify that getHookPermissions supports enabledBlocksBitmap, so a pool registered with a return-delta block on ProTradingPoolPresetHook fails every swap.	**FIXED**	Presets inherit HookRigOrchestrator.registerPoolStack -> _validateBitmapCompatibility, which checks the preset hook address flags (AccessControlBlock.sol:41-49).
S3-F43	90	**Buyer becomes currentKing with zero fee when waterfall cap is full** `KingOfThePoolBlock._processKingOfThePoolBuy` A buyer whose swap reaches the 2500 bps cap before KingOfThePool pays zero pot fee, yet processBuy still crowns that buyer as currentKing.	**BY-DESIGN**	The code comment at KingOfThePoolBlock.sol:151-153 explicitly crowns non-dust buys even when the 25% cap is already full and potContribution==0.
S3-F44	90	**Zero potContribution in processBuy still crowns the buyer and resets deadline** `KingOfThePoolBlock.processBuy` KingOfThePoolLogic.processBuy updates currentKing even when potContribution is zero, so a buyer can claim the pot without a fee contribution.	**PARTIAL**	Dust buys can no longer be crowned (line 156 requires a nominal fee >0). Zero-contribution crowning remains for non-dust buys when the cap is full or the pot is in another currency (lines 154-156). → TODO-28

ID	Conf.	Finding	Status	Current evidence
S3-F45	90	**processWaterfallKing calls processBuy even when potCut rounds to zero** `KingOfThePoolBlock.processWaterfallKing` processWaterfallKing calls processBuy even when potCut is zero, so an attacker can become currentKing and win the jackpot with zero contribution.	**FIXED**	processBuy line 154-156 crowns only if potContribution>0 or the nominal potFee mulDiv is >0, so rounding-to-zero dust buys are not crowned.
S3-F46	90	**Partial tick traversal in _processCrossedLimitOrderTicks inverts reverse swap direction** `LimitOrdersBlock._processCrossedLimitOrderTicks` LimitOrdersLogic.processCrossedTicks stores a partial tick after 512 steps, so a reverse swap reads the wrong direction and fills opposite limit orders.	**FIXED**	processCrossedTicks always stores `lastTicks[poolId] = postSwapTick` (L373), not a partial tick, so direction cannot invert.
S3-F47	90	**Capped 512-tick loop in processCrossedTicks desynchronizes lastTicks** `LimitOrdersBlock.processCrossedTicks` When a swap crosses more than 512 ticks, processCrossedTicks stores an incomplete lastTicks value, and a reversal swap skips valid limit orders.	**FIXED**	lastTicks set to postSwapTick (L373), so no desync. Buckets beyond 512 steps in that swap are still skipped (see S3-L05).
S3-F48	90	**Lagging lastTicks after 512 steps inverts movedUp on the next swap** `LimitOrdersLogic.processCrossedTicks` A swap across more than 512 ticks leaves lastTicks behind postSwapTick, so the next swap reverses movedUp and fills opposite limit orders.	**FIXED**	No lagging lastTicks: L373 writes postSwapTick unconditionally after the capped loop.
S3-F49	90	**Oracle checkpoint integrates pre-swap tick instead of post-swap tick** `OracleBlock._updateOracleCheckpoint` updateCheckpoint integrates obs.lastTick instead of the current pool tick over elapsed time, so the oracle reports the price from before the last swap.	**FIXED**	updateCheckpoint integrates the passed currentTick, not obs.lastTick (OracleBlock.sol:82-86); call site passes the live pre-swap tick.
S3-F50	90	**One-wei-per-second dust orders fill all 256 active TWAMM slots for a year** `TWAMMBlock.submitTWAMMOrder` submitOrder accepts one wei per second up to 256 active orders, so an attacker can fill all order slots with dust and block new orders.	**OBSOLETE**	submitTWAMMOrder reverts (TWAMMBlock.sol:270-272), so the 256 slots cannot be filled.
S3-F51	90	**Uint160 cast before MAX_SQRT_PRICE check wraps graduation price** `VirtualXYKLaunch._graduateToV4` _graduateToV4 casts the shifted square root to uint160 before the MAX_SQRT_PRICE check, so a high token-to-quote ratio wraps the price and makes graduation revert.	**FIXED**	Tiered branches bound the sqrt before the cast (X160 for tokens<=uint96; X128 overflows in mulDiv before a uint160 wrap) (VirtualXYKLaunch.sol:201-216).
S3-F52	90	**High token-to-quote ratio overflows liquidity math in VirtualXYKLaunch.unlockCallback** `VirtualXYKLaunch.unlockCallback` A launch with a high token-to-quote ratio overflows the uint160 cast in _graduateToV4 and fails to settle in unlockCallback.	**FIXED**	Tiered sqrt computation prevents the uint160 wrap (VirtualXYKLaunch.sol:201-216). sqrt(quote*tokens) overflows only beyond realistic supply.
S3-F53	90	**Initial sell in unlockCallback leaves transient balance at zero** `WalletCapBlock._enforceWalletCapAfterSwap` WalletCapLogic.enforceAfterSwap subtracts sold tokens from zero instead of actualBal in TransientContext, so a trader who sells then buys in one unlockCallback reverts.	**FIXED**	WalletCapBlock.sol:96-99: when the cached value is zero, a sell seeds prevExpected from balanceOf(trader) before subtracting sold.

ID	Conf.	Finding	Status	Current evidence
S3-F54	90	**Sell before buy in one unlock callback triggers false WalletCapExceeded revert** `WalletCapBlock.enforceAfterSwap` WalletCapLogic.enforceAfterSwap subtracts sold tokens from zero instead of the token balance on an initial sell, so a subsequent buy in the same transaction reverts.	**FIXED**	Same fix (WalletCapBlock.sol:97-98). The cached value is balanceOf minus sold, not zero, and enforceAfterBuy takes max(actual, cached), so no false revert.
S3-F55	85	**Floor reserve donation via PoolManager.donate lets flash liquidity take reserves** `AutoBuybackBlock.processWaterfallBuyback` AutoBuybackBlock donates reserves via PoolManager.donate, so an attacker who adds flash liquidity at triggerFloorTick takes the reserve in one transaction.	**FIXED**	Donations are queued and flushed in beforeSwap of a later swap, skipped if liquidity was added this block (AutoBuybackBlock.sol:146, Orchestrator:277-279). Same-tx flash liquidity is blocked.
S3-F56	85	**Intra-block swaps overwrite lastObservedTick and dilute dynamic fee volatility** `DynamicFeeBlock._computeDynamicFee` A trader who splits a large trade into smaller swaps in one block advances lastObservedTick each time and lowers the dynamic fee.	**FIXED**	compute only updates lastObservedTick on a new block. Same-block swaps only raise rollingVolatilityTicks (DynamicFeeBlock.sol:93-99).
S3-F57	85	**Three-tick swaps advance lastObservedTick while volatility rounds to zero** `DynamicFeeBlock.compute` A trader who splits a swap into three-tick steps updates state.lastObservedTick while state.rollingVolatilityTicks rounds down to zero, so the trader pays baseFee.	**FIXED**	The EMA adds +3 when absDelta>0 (`((r*3 + absDelta + 3)/4)`), so three-tick moves no longer round volatility to zero (DynamicFeeBlock.sol:94).
S3-F58	85	**Standard pools lack a caller check in _beforeInitialize** `HookRigOrchestrator._beforeInitialize` The _beforeInitialize hook checks the caller only for launchpad pools, so an attacker can initialize any standard pool at a wrong price.	**PRESENT**	_beforeInitialize (HookRigOrchestrator.sol:203) still checks authorizedCurve/factory only for launchpad pools; there is no caller check for standard pools. → TODO-23
S3-F59	85	**Swap rewards use raw unspecified token units across swap modes** `HookRigOrchestrator._runAfterSwapPipeline` _runAfterSwapPipeline passes raw unspecifiedCurrency volume to _accrueSwapRewards, so a trader chooses the swap mode with the cheaper token to inflate reward points.	**FIXED**	HookRigOrchestrator.sol:409-411: reward volume is always the project-token leg (unspecified minus cut, or settledSpecifiedAmount), whatever the swap mode.
S3-F60	85	**Zero maxWalletBps default in _validateWalletCapAlignment mismatches WalletCapLogic** `LaunchpadFactory._validateWalletCapAlignment` LaunchpadFactory._validateWalletCapAlignment treats a zero maxWalletBps as 200 bps while WalletCapLogic.init stores zero, so the pool disables the wallet cap after graduation.	**FIXED**	_validateWalletCapAlignment writes maxWalletBps=200 into stackConfig when it is zero, before registerPoolStack (LaunchpadFactory.sol:224-226).
S3-F61	85	**One-tick swap locks opposite-direction swaps for all users in that block** `MEVProtectionBlock._enforceMEVAfterSwap` enforceAfterSwap blocks every opposite-direction swap in the same block globally, so an attacker can make a one-tick swap first to block opposite trades.	**PARTIAL**	MEVProtectionBlock.sol:74-85: only swaps of at least dirThreshold (max(maxDelta/20,2) ticks) set the direction, so one-tick swaps cannot; a threshold-sized swap still blocks opposite swaps. → TODO-28
S3-F62	85	**Pool-wide lastSwapZeroForOne in enforceAfterSwap blocks opposite swaps in the same block** `MEVProtectionBlock.enforceAfterSwap` An attacker who front-runs with a one-tick swap sets lastSwapZeroForOne, which makes every opposite swap in that block revert in MEVProtectionLogic.enforceAfterSwap.	**PARTIAL**	The pool-wide lastSwapZeroForOne is set only by swaps of at least dirThreshold ticks (MEVProtectionBlock.sol:74-85); an attacker paying that impact still blocks opposite trades that block. → TODO-28

ID	Conf.	Finding	Status	Current evidence
S3-F63	85	**Unallocated band tokens and remainders stay locked in NativeTickLaunch** `NativeTickLaunch.seedLiquidity` LaunchpadFactory sends the full available token supply to NativeTickLaunch, but seedLiquidity leaves unallocated band tokens and division remainders locked in the contract forever.	PRESENT	Excess supply sent by the factory (Factory:333-334) and skipped or unused band tokens stay in NativeTickLaunch, which has no sweep (unlockCallback:72-103). → TODO-27
S3-F64	85	**Base-price redemption cap in SteppedTrancheLaunch.sell locks surcharges and step premiums** `SteppedTrancheLaunch.sell` A buyer who exits a stalled SteppedTrancheLaunch via sell loses all anti-snipe surcharges and tranche price increments, which stay locked in the contract forever.	PRESENT	sell caps at basePriceQ96/pro-rata (Stepped:193-196). totalSurchargeCollected has no refund or withdraw path in a stalled launch. → TODO-26
S3-F65	85	**Leftover reserve donation at graduation locks tokens inside PoolManager** `SteppedTrancheLaunch.unlockCallback` SteppedTrancheLaunch and VirtualXYKLaunch pass leftover reserve tokens to poolManager.donate at graduation, which locks those tokens as unclaimable fees inside PoolManager.	BY-DESIGN	Leftover donation in unlockCallback (Stepped:295-307) is an accepted design limit: locked graduation LP fees stay in PoolManager (docs/04:91, docs/07 TODO-21).
S3-F66	85	**Integer division remainder in TWAMMLogic.submitOrder is locked forever** `TWAMMLogic.submitOrder` TWAMMLogic.submitOrder rounds the per-second rate down and cancelOrder rejects expired orders, so the hook locks the remainder totalAmount modulo durationSeconds forever.	OBSOLETE	submitOrder is unreachable (submit disabled, L270-272). Cancel refunds full totalAmount even after expiry, so there is no remainder lock (L126-153).
S3-F67	85	**Leftover tokens donated in VirtualXYKLaunch.unlockCallback are locked as unclaimable fees** `VirtualXYKLaunch.unlockCallback` VirtualXYKLaunch and SteppedTrancheLaunch pass leftover tokens to poolManager.donate on a locked LP position, so unsold tokens become unclaimable fees.	BY-DESIGN	VirtualXYKLaunch.unlockCallback:252-264 donates leftovers to the locked LP, documented as accepted (docs/04:91; docs/07 TODO-21).
S3-F68	80	**Window rollover sets anchorTick from unverified instantaneous preSwapTick** `CircuitBreakerBlock.checkAndAdvance` checkAndAdvance sets anchorTick to the instantaneous preSwapTick when a window expires, so an attacker can lock anchorTick at an extreme tick.	PARTIAL	The rollover anchor still comes from the instantaneous preSwapTick, but is clamped to prevAnchor+/-maxDev, so an extreme lock is bounded to one band. → TODO-28
S3-F69	80	**VirtualXYKLaunch.buy credits full quoteUsed without a fee-on-transfer balance check** `VirtualXYKLaunch.buy` buy credits the full quoteUsed to realQuoteRaised without a balance check, so a fee-on-transfer quoteCurrency leaves the curve insolvent and blocks _graduateToV4.	PRESENT	buy adds quoteUsed to realQuoteRaised after safeTransferFrom with no balance-delta check (VirtualXYKLaunch.sol:117,122). → TODO-17
S3-L01	lead	**Floor defense donates reserves to LPs instead of a buyback swap** `AutoBuybackBlock._processAutoBuyback` Code smells: AutoBuybackLogic.settleLPDonation calls manager.donate(key, amount0, amount1, "") instead of a pool swap, which increases LP fee growth without moving sqrtPriceX96 or tick. — AutoBuybackLogic.settleLPDonation calls PoolManager.donate instead of a swap below triggerFloorTick. We did not verify whether an external con…	BY-DESIGN	flushPendingDonations calls manager.donate (AutoBuybackBlock.sol:154). docs/04 line 123 and section 3.7 document floor defense as a queued LP donation, not a swap.

ID	Conf.	Finding	Status	Current evidence
S3-L02	lead	**Router unlockCallback settles native ETH from contract balance without msg.value check** `HoodiSwapRouter.unlockCallback` Code smells: HoodiSwapRouter.unlockCallback in script/HoodiIntegration.s.sol calls data.key.currency0.settle without verifying msg.value against -delta.amount0(). — HoodiSwapRouter.unlockCallback settles native ETH from the router balance without a msg.value check. We did not verify if the router holds ETH between transactions.	PARTIAL	unlockCallback still settles native ETH without checking msg.value, so any ETH stranded in the router (e.g. via receive()) could be spent. Excess is now refunded, so it normally holds none. → TODO-20
S3-L03	lead	**Configured jitExitPenaltyBps is never deducted on liquidity removal** `JITGuardLogic.enforceOnRemove` Code smells: JITGuardLogic.enforceOnRemove is a view function and HookRigOrchestrator sets afterRemoveLiquidityReturnDelta = false, so jitExitPenaltyBps only forces minHold = 1. — JITGuardLogic.enforceOnRemove never deducts jitExitPenaltyBps on liquidity removal. Same-block removals revert instead when minLPHoldBlocks is zero.	PARTIAL	enforceOnRemove is still a view that never deducts jitExitPenaltyBps (JITGuardBlock.sol:34-63); the penalty only forces minHold=1; docs/04:53,119 claim it is enforced. → TODO-16
S3-L04	lead	**Tick loop includes prevTick when prevTick is aligned to tickSpacing** `LimitOrdersBlock._processCrossedLimitOrderTicks` Code smells: LimitOrdersLogic.processCrossedTicks starts at aligned = (prevTick / step) * step, which equals prevTick when prevTick % step == 0. — The tick loop checks prevTick when prevTick is a multiple of tickSpacing. We did not verify an active override of _swapFilledOrderOnClaim.	PRESENT	Loop still starts at aligned==prevTick (LimitOrdersBlock.sol:349,361). Likely benign: price leaving t crosses t, and re-marking is idempotent. → TODO-20
S3-L05	lead	**Large swap followed by reverse swap skips buckets beyond 512 ticks** `LimitOrdersBlock._processLimitOrderTicks` Code smells: LimitOrdersLogic.processCrossedTicks caps the loop at MAX_TICK_FILLS_PER_SWAP = 512 and stores lastReached in lastTicks[poolId]. — A large swap followed by a reverse swap skips limit order buckets beyond 512 ticks. We did not verify the gas cost for 512 ticks.	PARTIAL	Reverse-swap desync fixed (L373). Buckets beyond MAX_TICK_FILLS_PER_SWAP=512 steps in one swap are still never marked filled (L351,363). Not documented. → TODO-20
S3-L06	lead	**Virtual order execution advances timestamp without a swap of maker tokens** `TWAMMBlock._executeTWAMM` Code smells: TWAMMBlock._executeTWAMM advances lastVirtualOrderTimestamp and emits TWAMMVirtualOrdersExecuted without calling poolManager.swap or crediting maker output. — Every swap advances lastVirtualOrderTimestamp in TWAMMLogic.executePending without a swap of the locked maker tokens. No claim function exists in TWAMMBlock.	OBSOLETE	executePending still records virtual volume without swapping (TWAMMBlock.sol:206-210). No orders can exist (submit disabled), and docs/04:117 documents this.

8. Scan 4 — 2026-10-15 10:03 (594edcd)

3-pass scan of commit `594edcd` (2026-10-15 3-pass loop). Remediation: working tree (uncommitted, this session). 35 findings and 22 leads; current status: 32 fixed, 2 mitigated, 3 by-design, 2 partial, 18 pending.

ID	Conf.	Finding	Status	Current evidence
S4-F01	100	**Order maker re-enters PoolManager.swap while every hook check is off** `HookRigOrchestrator._beforeSwap` · seen in 3/3 runs · NEW The internal-swap flag covers the whole pool and `claimLimitOrder` has no reentrancy guard, so a maker who gets the ETH payout swaps with no fee or check.	FIXED	Internal-execution flag removed; claim swaps run the full hook pipeline (HookRigOrchestrator._beforeSwap/_afterSwap); claimLimitOrder is nonReentrant. Test test_Fix_ClaimSwapPaysCutsAndCreditsMaker.
S4-F02	95	**Claim swap pays no LP fee and skips every hook cut** `LimitOrdersBlock.claimLimitOrder` · seen in 3/3 runs · NEW The claim swap returns fee 0 with no override flag, so on a dynamic-fee pool the maker pays a zero LP fee and no hook cut.	FIXED	Claim swap pays the resolved LP fee and every return-delta cut; executor reports the maker via IMsgSender so fees/points/crowns go to the maker.
S4-F03	95	**The daily sell cap reads a balance the seller controls** `VestingSellCapBlock.enforceSellCap` · seen in 3/3 runs · KNOWN (2 scans) The epoch balance comes from `balanceOf(seller)` at the first sell, so router sells revert and a seller who borrows or moves in tokens first sells far past the cap.	FIXED	_preSellBalance nets PoolManager deltas; snapshot lowered when tokens leave; zero-balance untrusted routers skipped instead of bricked. Tests test_Fix_VestingSellCap_*.
S4-F04	90	**First buyer after a stepped graduation keeps the crown and takes the seeded pot** `KingOfThePoolBlock.processBuy` · seen in 2/3 runs · KNOWN (2 scans) The launch seed puts the pot in the quote currency, so later project-token buys add no pot and cannot take the crown.	FIXED	Non-dust buys take the crown whenever a pot exists, even with zero pot cut; dust (nominal fee 0) never crowns. Test test_Fix_KingSeedInOtherCurrencyStillContestable.
S4-F05	90	**Attacker copies a pending launch salt and takes the announced token address** `LaunchpadFactory._deployTokenAndVest` · seen in 1/3 runs · NEW The CREATE2 salt does not include `msg.sender`, so an attacker who copies a pending launch deploys the token first with the attacker's pool admin, fee recipients and curve settings.	FIXED	CREATE2 salt = keccak256(msg.sender, salt) in _deployTokenAndVest and predictTokenAddressFor. Test test_Fix_LaunchSaltIsBoundToSender.
S4-F06	85	**Exact-output buys skip the auto burn and the King pot fee** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` · seen in 1/3 runs · KNOWN (2 scans) The burn and King steps require the project token as the unspecified currency, so a buyer who uses exact output pays neither cut.	FIXED	Exact-output buys revert ExactOutputBuyNotSupported on AutoBurn/King pools (AfterSwapDeltaAggregator).
S4-F07	85	**Attacker registers the predicted launch pool first and blocks the launch** `HookRigOrchestrator.registerPoolStack` · seen in 1/3 runs · KNOWN (2 scans) `initPoolMeta` checks the factory only when `authorizedCurve` is set, so an attacker who registers the predicted launch pool makes the launch revert and holds its pool admin role.	FIXED	initPoolMeta rejects pools whose tokens have no code (TokenNotDeployed), so a predicted launch pool cannot be pre-registered. Test test_Fix_RegisterPoolStackRejectsUndeployedToken.
S4-F08	85	**Add from one address and remove from another skips the JIT hold** `JITGuardBlock.recordAddition` · seen in 1/3 runs · KNOWN (2 scans) For salt 0 the guard key includes the resolved user, so an attacker adds through a trusted router as one address and removes as another in the same block.	PENDING	Not changed: keying the JIT hold by position lets router users re-lock each other; bypass also works with two positions. Tracked as TODO-16. → TODO-16

ID	Conf.	Finding	Status	Current evidence
S4-F09	85	**Creator launches with a custom hook that blocks buyer sells** `LaunchpadFactory.launchVirtualXYK` · seen in 1/3 runs · KNOWN (2 scans) The factory accepts any `hookContract`, so a creator graduates the curve into a pool whose hook rejects buyer sells, and the creator sells into the locked LP.	**FIXED**	_resolveHook accepts only defaultOrchestrator (UnsupportedHookContract). Test test_Fix_LaunchRejectsUnsafeStacksAndHooks.
S4-F10	85	**Claim swap has no price limit and no minimum output** `LimitOrdersBlock.claimLimitOrder` · seen in 2/3 runs · NEW The executor swaps with the widest price limit and checks only `outAmount > 0`, so an attacker who trades around the claim keeps most of the maker output.	**FIXED**	New claimLimitOrder(key, orderId, minAmountOut) overload reverts ClaimOutputBelowMinimum. Test test_Fix_ClaimSwapMinimumOutput.
S4-F11	80	**Ending a swap outside all ranges skips the ArbRecapture cut** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` · seen in 1/3 runs · NEW The hook checks liquidity only at the post-swap tick, so an arbitrageur who ends one tick past the last range pays no ArbRecapture or LPRewards cut.	**FIXED**	ArbRecapture and LPRewards cuts no longer depend on post-swap liquidity; they are always taken and queued.
S4-F12	80	**A trader takes back the ArbRecapture and LPRewards donation** `AfterSwapDeltaAggregator._executeAfterSwapDeltaWaterfall` · seen in 3/3 runs · NEW The cuts are donated to in-range liquidity inside afterSwap, so a trader who adds narrow liquidity at the post-swap tick in the same unlock gets most of them back.	**FIXED**	LP donations queued (pendingLPDonation) and flushed in beforeSwap of a later swap in a block with no liquidity add. Test test_Fix_LPDonationWaitsForBlockWithoutLiquidityAdds.
S4-F13	80	**One reserve threshold for both currencies locks the other reserve forever** `AutoBuybackBlock.accumulateAndCheckTrigger` · seen in 1/3 runs · NEW The trigger compares both currency reserves with one raw `minBuybackReserve`, so the reserve in the smaller-unit currency never triggers and no function releases it.	**FIXED**	minBuybackReserve is quote-denominated; on trigger both currency reserves move into the queued donation.
S4-F14	80	**Narrow LP added one transaction earlier takes the buyback donation** `AutoBuybackBlock.accumulateAndCheckTrigger` · seen in 1/3 runs · NEW The only anti-JIT guard uses transient storage, so an attacker adds narrow liquidity in one transaction and triggers the floor donation in the next to take most of the reserve.	**FIXED**	Transient same-tx guard replaced by a block-scoped _lastLiquidityAddBlock check before any donation is paid.
S4-F15	80	**Preset hooks accept blocks whose liquidity callbacks they never receive** `AccessControlLogic.validateBitmapAndPermissions` · seen in 3/3 runs · KNOWN (2 scans) The validator uses a wrong full-flags constant and checks only two flags, so JITGuard, AccessControl and the AutoBuyback same-transaction guard never run on preset hooks.	**FIXED**	validateBitmapAndPermissions requires the hook address to carry every flag permissionsForBitmap(bitmap) needs. Test test_Fix_PresetHookRejectsBlocksWithoutCallbacks.
S4-F16	80	**Creator enables AccessControl so launch buyers cannot sell** `LaunchpadFactory._deploySortedTokenAndRegisterPool` · seen in 1/3 runs · NEW The factory accepts a stack with AccessControl and an empty root, so after graduation only the creator can sell into the locked LP.	**FIXED**	Factory rejects BIT_ACCESS_CONTROL on launch stacks (UnsafeLaunchStack).

ID	Conf.	Finding	Status	Current evidence
S4-F17	80	**Creator restarts an 80% anti-snipe fee at graduation and takes holder sales** `LaunchpadFactory._deploySortedTokenAndRegisterPool` · seen in 1/3 runs · NEW The anti-snipe fee is outside the 25% budget and restarts at graduation, so a creator with narrow liquidity takes most of each holder's sale.	FIXED	Factory caps AntiSnipe launchFee at MAX_LAUNCH_ANTI_SNIPE_FEE = 25% on launch stacks.
S4-F18	80	**A tick spacing above 32767 makes graduation revert forever** `LaunchpadFactory._deploySortedTokenAndRegisterPool` · seen in 1/3 runs · NEW The factory checks only the lower bound of `tickSpacing`, so every graduating buy reverts in `PoolManager.initialize` and buyer quote stays in the curve.	FIXED	Factory reverts when tickSpacing > TickMath.MAX_TICK_SPACING for every curve type.
S4-F19	80	**A 1-wei first swap blocks opposite trades for the block** `MEVProtectionBlock.enforceAfterSwap` · seen in 1/3 runs · KNOWN (2 scans) The tracker records the direction of any first swap in a block, so an attacker's 1-wei swap makes larger opposite trades revert for that block.	FIXED	Only swaps of at least dirThreshold set the block direction (blockDirectionSet). Test test_Fix_MEVDustSwapDoesNotSetBlockDirection.
S4-F20	80	**A rounding overshoot opens the stepped pool at the next tranche price** `SteppedTrancheLaunch._graduateToV4` · seen in 1/3 runs · NEW The final buy sells a few wei past the tranche boundary, so graduation prices the pool at the next tranche and the pool opens 20% above the price buyers paid.	FIXED	Graduation opens at lastFillPriceQ96 (price of the last curve fill), not the rounded tranche index. Fuzz repro GL-11 passes.
S4-F21	80	**Graduation clamps LP tokens but keeps all quote, so the pool opens above the curve price** `VirtualXYKLaunch._graduateToV4` · seen in 1/3 runs · NEW When `virtualTokenReserve` exceeds the real token supply, graduation seeds fewer tokens with the full quote, so an early buyer sells above the curve price and takes LP quote.	FIXED	When LP tokens are clamped, quoteToSeed is scaled proportionally so the pool opens at the curve price.
S4-F22	80	**The curve LP position cannot collect its fees** `VirtualXYKLaunch.unlockCallback` · seen in 2/3 runs · NEW Traders pay LP fees and LP-reward donations to the curve's position, but no function collects them, so the tokens stay in PoolManager.	BY-DESIGN	Curve-owned graduation LP is locked by design (equivalent of burned LP); documented in docs/04 section 3.7 and TODO-21.
S4-F23	75	**AutoBuyback does not request beforeAddLiquidity** `HookPermissionMasks.permissionsForBitmap` · seen in 1/3 runs · KNOWN (2 scans) The `beforeAddLiquidity` mask leaves out `BIT_AUTO_BUYBACK`, so on preset hooks an LP adds liquidity and takes the floor donation in one transaction.	FIXED	permissionsForBitmap requests beforeAddLiquidity for AutoBuyback/ArbRecapture/LPRewards and wallet-rule blocks, and beforeSwap for donation blocks.
S4-F24	75	**One-sided liquidity lets traders skip the sell cap, buy cap and taxes** `HookRigOrchestrator._beforeAddLiquidity` · seen in 2/3 runs · NEW A trader uses a one-sided range order instead of a swap, so the vesting cap, the wallet cap and the buy and sell cuts never run.	FIXED	enforceTwoSidedLiquidity rejects one-sided adds on WalletCap/VestingSellCap pools (curve exempt). Test test_Fix_WalletCapPoolRejectsOneSidedLiquidity.
S4-F25	75	**Pool admin takes the anti-snipe surcharge when King is off** `HookRigOrchestrator.seedKingPotFromLaunchCurve` · seen in 2/3 runs · KNOWN (2 scans) The seed falls back to the pool admin's claim when the pool has no King block, so the admin takes the buyers' surcharge.	FIXED	seedPotFromLaunchCurve never credits the pool admin; SteppedTrancheLaunch seeds the pot only when King is enabled, otherwise all surcharge goes to LP.

ID	Conf.	Finding	Status	Current evidence
S4-F26	75	**A pool admin raises the tax from 0% to 10% after buyers hold the token** `HookRigOrchestrator.setBuySellTax` · seen in 1/3 runs · NEW `decayOnly` is optional and the factory never sets it, so the admin raises the tax after launch and takes 10% of every swap.	FIXED	BuySellTaxLogic.update forbids any tax increase regardless of decayOnly. Test test_Fix_TaxCannotIncreaseWithoutDecayOnly.
S4-F27	75	**ArbRecapture fills the cut cap so large buyers never take the crown** `KingOfThePoolBlock.processWaterfallKing` · seen in 1/3 runs · NEW The tax budget does not count the ArbRecapture cut, so a large buy fills the 25% cap before the King step and the current king keeps the pot.	FIXED	Crown rule no longer needs a pot cut (non-dust buys crown when a pot exists). ArbRecapture is still not counted in validateTaxBudget (TODO-20).
S4-F28	75	**A zero pot fee locks the launch seed in the hook** `KingOfThePoolBlock.seedPotFromLaunchCurve` · seen in 2/3 runs · NEW The curve seeds the King pot when the pot fee is zero, so no buyer ever becomes king and the seeded quote stays locked in the hook.	FIXED	initKing requires potFeeOnBuyBps > 0; seeds into a zero-fee pot are impossible. Test test_Fix_KingRequiresPotFee.
S4-F29	75	**launchNativeTick skips the wallet-cap default** `LaunchpadFactory.launchNativeTick` · seen in 1/3 runs · NEW `launchNativeTick` does not call `_validateWalletCapAlignment`, so a zero `maxWalletBps` turns off the wallet cap and the first buyer can take the whole band supply.	FIXED	launchNativeTick calls _validateWalletCapAlignment(stackConfig, 0) so a zero cap defaults to 200 bps.
S4-F30	75	**NativeTick band positions cannot collect their fees** `NativeTickLaunch.unlockCallback` · seen in 1/3 runs · NEW The curve owns every band position and has no collect path, so all LP fees, including the anti-snipe fee, stay in PoolManager.	BY-DESIGN	Curve-owned NativeTick band positions are locked by design; documented in docs/04 section 3.7 and TODO-21.
S4-F31	75	**The oracle weights each interval by the previous interval's tick** `OracleBlock.updateCheckpoint` · seen in 1/3 runs · KNOWN (2 scans) The checkpoint multiplies the elapsed time by the old `lastTick`, so an attacker who moves the price for one block skews the TWAP for the whole next interval.	FIXED	Cumulative adds clamped current pre-swap tick x elapsed; live extrapolation uses the clamped slot0 tick. Test test_Fix_OracleWeightsIntervalByItsOwnTick.
S4-F32	75	**Dust TWAMM orders fill the per-pool order cap for a year** `TWAMMBlock.submitTWAMMOrder` · seen in 1/3 runs · NEW The 256-order cap is shared and has no real minimum size, so an attacker opens 256 dust orders and blocks every other TWAMM order on the pool.	MITIGATED	submitTWAMMOrder reverts TWAMMOrdersDisabled, so the order cap cannot be squatted. Real execution tracked as TODO-14.
S4-F33	70	**TWAMM deposits never swap and have no output claim** `TWAMMBlock.submitTWAMMOrder` · seen in 2/3 runs · KNOWN (2 scans) `executePending` only counts virtual volume, so a TWAMM user never receives output tokens.	MITIGATED	New TWAMM orders disabled so no deposit can be stranded; existing orders refund in full. Real execution tracked as TODO-14.
S4-F34	65	**Moving tokens to new wallets resets the daily sell cap** `VestingSellCapBlock._enforceVestingSellCap` · seen in 1/3 runs · NEW The epoch is keyed by the selling address, so a holder who moves the rest to a new wallet after each sell sells most of the holding in one day.	BY-DESIGN	Per-wallet caps cannot bind a holder who splits tokens across wallets; documented in docs/04 section 3.7 and TODO-21.

ID	Conf.	Finding	Status	Current evidence
S4-F35	65	**Batched buys on two pools of one token get twice the wallet cap** `WalletCapBlock.enforceAfterBuy` · seen in 1/3 runs · NEW The wallet-cap cache is keyed by pool and not by token, so a buyer who batches buys on two pools of one token gets twice the cap.	FIXED	Wallet-cap transient cache keyed by project token instead of PoolId, so batched buys across pools of one token add up.
S4-L01	lead	**An allowlisted untrusted router admits every caller** `AccessControlBlock.verifyAccess` Code smells: the check uses the router address, not the user — We did not confirm a deployment that allowlists such a router.	PENDING	AccessControl checks the router address for untrusted routers, so an allowlisted public router admits every caller (TODO-20). → TODO-24
S4-L02	lead	**A short burn take spends other users' escrow** `AutoBurnBlock.execute` Code smells: the burn uses the hook's shared ERC20 balance — We did not find a token that causes the shortfall.	PENDING	AutoBurn burns from the hook's shared ERC20 balance; an under-delivering take would spend escrow (TODO-20). → TODO-20
S4-L03	lead	**The deploy script falls back to a public Anvil key** `HoodiIntegrationScript.run` Code smells: `vm.envOr("PRIVATE_KEY", <Anvil key #0>)` with `--broadcast` on Hoodi — Anyone who knows that key owns the orchestrator. We did not confirm a broadcast with that key.	PENDING	Deploy script falls back to the public Anvil key #0 (TODO-18). → TODO-18
S4-L04	lead	**The script router pays native input from its own ETH** `HoodiSwapRouter.swap` Code smells: `swap` does not check that `msg.value` covers the input — Impact needs ETH that already sits in the router.	PARTIAL	Script router refunds excess, but unlockCallback can still spend ETH already held by the router (script-only; TODO-20). → TODO-20
S4-L05	lead	**Per-pool claims draw on one shared ERC-6909 balance** `HookRigBase.withdrawClaim` Code smells: no per-pool backing total — Any unbacked credit in one pool burns other pools' backing. We found only the owner path.	PENDING	Per-pool claims share one ERC-6909 balance; only an owner-set factory can create unbacked credit (TODO-15). → TODO-15
S4-L06	lead	**Exact-output sells earn fewer swap points** `HookRigOrchestrator._runAfterSwapPipeline` Code smells: volume is `gross - cut` where the trader pays `gross + cut` — The same sale earns different points by mode. We found no payout that uses the points.	PENDING	Exact-output sells earn swap points on gross minus cut instead of gross plus cut (TODO-20). → TODO-20
S4-L07	lead	**A launchpad factory address set by the owner can credit unbacked claims** `HookRigOrchestrator.seedKingPotFromLaunchCurve` Code smells: no ERC-6909 backing check; `launchpadFactory` can be changed at any time — The owner can withdraw other pools' claims. The deploy script's public fallback key may make this reachable by anyone; we did not confirm a live deployment with that key.	PENDING	Owner can point launchpadFactory at an address that credits unbacked King pots (TODO-15). → TODO-15
S4-L08	lead	**A 20-wei buy takes the crown when the minimum is zero** `KingOfThePoolBlock.initKing` Code smells: `minBuyAmountForReset = 0` is accepted — Any buyer can take the crown with dust just before the deadline. We did not show this is outside the game design.	PENDING	minBuyAmountForReset = 0 lets small non-dust buys crown just before the deadline (creator configuration; TODO-20). → TODO-28
S4-L09	lead	**The seed credit does not check that the hook received ERC-6909** `KingOfThePoolBlock.seedPotFromLaunchCurve` Code smells: the function trusts `launchpadFactory` for every pool — Only an owner-set factory reaches this. The shipped curves always mint before they seed.	PENDING	seedPotFromLaunchCurve does not verify the minted ERC-6909 backing (TODO-15). → TODO-15

ID	Conf.	Finding	Status	Current evidence
S4-L10	lead	**A zero creator makes the factory the pool admin** `LaunchpadFactory._deploySortedTokenAndRegisterPool` Code smells: admin falls back to `msg.sender`, the factory — The surcharge seed goes to a claim nobody can withdraw. We did not prove a third party profits.	PENDING	A zero-creator launch makes the factory the pool admin (TODO-20). → TODO-29
S4-L11	lead	**launchVirtualXYK accepts a tick spacing the PoolManager rejects** `LaunchpadFactory.launchVirtualXYK` Code smells: only values at or below 0 are replaced — Buyers can only exit through `sell`. We did not find a party who gains.	FIXED	Factory rejects tickSpacing > MAX_TICK_SPACING for every launch type.
S4-L12	lead	**A maker can neither claim nor cancel with no liquidity in range** `LimitOrdersBlock.claimLimitOrder` Code smells: a zero-output claim reverts and cancel rejects filled buckets — We did not prove a lock longer than the time until new liquidity arrives.	FIXED	Makers can cancel filled orders, so a zero-output claim can no longer lock funds.
S4-L13	lead	**A 1-wei order marks a filled bucket unfilled** `LimitOrdersBlock.placeLimitOrder` Code smells: `placeOrder` resets `filled` when the price is back on the far side — Earlier makers cannot claim until the price crosses again. They can still cancel.	PARTIAL	A 1-wei order can still clear a filled bucket and delay claims; cancel always refunds (TODO-20). → TODO-20
S4-L14	lead	**A round-trip swap marks a bucket filled** `LimitOrdersBlock.processCrossedTicks` Code smells: any crossing sets `filled`, even if the price comes back in the same transaction — Makers then lose cancel. We did not show a profit for the attacker.	FIXED	A round-trip fill no longer removes the maker's cancel path (cancel works after fill).
S4-L15	lead	**zeroForOne orders fill when the price falls** `LimitOrdersBlock.processCrossedTicks` Code smells: downward crossings fill token0-input buckets — The order acts as a stop order. We did not confirm the intended design.	PENDING	zeroForOne orders fill on a downward crossing (stop-order semantics); intended behaviour not yet confirmed (TODO-20). → TODO-20
S4-L16	lead	**A large swap skips buckets after 512 ticks** `LimitOrdersBlock.processCrossedTicks` Code smells: the loop stops at 512 steps and then sets `lastTicks` to the post-swap tick — Orders past the cap stay unfilled. Makers can cancel, so we found no loss.	PENDING	A swap of more than 512 tick steps leaves later buckets unfilled; makers can cancel (TODO-20). → TODO-20
S4-L17	lead	**Surcharge quote can make the stepped pool open above the curve price** `SteppedTrancheLaunch._graduateToV4` Code smells: token amount is clamped, quote amount is not — We did not build numbers under the factory caps.	PENDING	SteppedTranche clamps LP tokens without scaling quote when surcharges are large (TODO-20). → TODO-26
S4-L18	lead	**Fee-on-transfer quote tokens make stepped graduation revert** `SteppedTrancheLaunch.buy` Code smells: `realQuoteRaised` adds the requested amount, not the received amount — We did not test a specific token.	PENDING	Fee-on-transfer quote tokens make SteppedTranche graduation revert (TODO-17). → TODO-17
S4-L19	lead	**A snapshot without afterSwap stays for later swaps** `TransientContext.storePreSwapSnapshot` Code smells: the snapshot clears only in afterSwap — Only a hook without afterSwap is affected. We found no such deployed hook.	PENDING	A hook without afterSwap would leave a stale pre-swap snapshot; no such hook ships (TODO-20). → TODO-20

ID	Conf.	Finding	Status	Current evidence
S4-L20	lead	**The init accepts sell caps outside the documented range** `VestingSellCapBlock._initVestingSellCap` Code smells: accepts 50 to 10_000 bps, docs say at most 5000 — We did not confirm the intended policy.	**PENDING**	VestingSellCap init accepts caps up to 10_000 bps vs the documented 5000 (TODO-20). → TODO-29
S4-L21	lead	**Fee-on-transfer quote tokens make graduation revert** `VirtualXYKLaunch.buy` Code smells: `realQuoteRaised` adds the requested amount, not the received amount — Graduation asks for more quote than the curve holds. We did not confirm a realistic quote token.	**PENDING**	Fee-on-transfer quote tokens make VirtualXYK graduation revert (TODO-17). → TODO-17
S4-L22	lead	**Untrusted routers let a buyer pass the wallet cap each transaction** `WalletCapBlock.enforceAfterBuy` Code smells: the check reads the router balance, which is zero at afterSwap — A buyer who uses an untrusted router can buy up to the cap in every transaction. We did not rate this above the transfer bypass.	**PENDING**	WalletCap checks the untrusted router balance, so each transaction can buy up to the cap (TODO-20). → TODO-24

9. Test Evidence & Disclaimer

Final state verified on 2026-10-15: `forge test` passes 87 local tests (unit, presets, launchpad, invariant, fuzz-harness Foundry tester and the new `AuditFixes` suite) and the 3 Hoodi fork integration tests, including the exhaustive run over all 2,843 valid Tier-3 block combinations. The fuzz-harness properties GL-11 and SP-09 were updated for the new graduation pricing and queued LP donations. Medusa/Echidna campaigns were not re-run after the scan-4 fixes.

This review was performed by AI agents. AI analysis can never verify the complete absence of vulnerabilities and no guarantee of security is given. Item statuses reflect an automated re-verification of the code on disk on 2026-10-15 and should be confirmed by a human reviewer. Team security reviews, bug bounty programs and on-chain monitoring are strongly recommended before mainnet deployment.